PROJECT REPORT (BEC-851)

On

MEDITRACK : A IOT BASED SMART MEDICATION BOX

Submitted for partial fulfillment of award of the degree of

Bachelor of Technology

In

Electronics and Communication Engineering

Submitted By

Robin Kr. Mandal (2201920310092)

Md. Zaid Adeeb (2201920310062)

Azlaan Anwar Khan (2201920310028)

Prashant Sharma (2201920310078)

Under the Guidance of

Dr. Vivek Gupta

Deptt. of Electronics and Communication Engineering

G. L. BAJAJ INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Plot no. 2, Knowledge Park III, Gr. Noida

(Even Sem-8th Semester)

Session: 2025-26

DECLARATION

We certify that

1. The work contained in this Project Report is original and has been done by us under the guidance of my supervisor.
2. The work has not been submitted to any other University or Institute for the award of any other degree or diploma.
3. We have followed the guidelines provided by the University in preparing the Report.
4. We have confirmed to the norms and guidelines in the Ethical Code of Conduct of the University.
5. Whenever we used materials (data, theoretical analysis, figures, and texts) from other sources we have given due credit to them by citing them in the text of the report and giving their details in the references. Further, we have taken permission from the copyright owners of the sources, whenever necessary.

Date:

Signature:

Name-Azlaan Anwar Khan

Roll No.-2201920310028

Signature:

Name- Prashant Sharma

Roll No.-2201920310078

Signature:

Name-Md.Zaid Adeeb

Roll No.-2201920310062

Signature:

Name- Robin kr Mandal

Roll No.-2201920310092

Deptt. of Electronics and Communication Engineering

G. L. BAJAJ INSTITUTE OF TECHNOLOGY AND MANAGEMENT

[Approved by AICTE, Govt. of India & Affiliated to A.K.T.U (Formerly U.P.T.U), Lucknow]

CERTIFICATE

Certified that **Azlaan Anwar Khan,Md Zaid Adeeb, Prashant Sharma,Robin kr Mandal** have carried out the project work presented in this report entitled **"MediTrack: IOT Based Smart Medication System"** for the Subject **Project II** (BEC-851) during the Academic session 2025-26 **(EVEN SEM).** The project embodies result of the work and studies carried out by Students and the contents of the report do not form the basis for the award of any other degree to the candidate or to anybody else.

(Dr. Vivek Gupta)
(Project Guide)
(Associate Professor)
Deptt. of ECE

(Dr. Shivesh Tripathi)
(Project Coordinator)
(Associate Professor)
Deptt. Of ECE

(Dr. Satyendra Sharma)
HOD,Deptt.of ECE

Date:

ACKNOWLEDGEMENT

We would like to take this opportunity to express our heartfelt gratitude to the individuals who have contributed significantly to the completion of this college project report.

First and foremost, we extend our sincere appreciation to **Dr. Satyendra Sharma**, Head of Department and **Dr Vivek Gupta** our project guide, for their exceptional guidance, expertise, and unwavering support throughout this project. Their valuable insights, constant encouragement, and dedication have been instrumental in shaping the successful outcome of this work.

Furthermore, I would like to acknowledge the invaluable assistance and coordination provided by **Dr. Shivesh Tripathi**, the Project Coordinator. Their meticulous planning, effective coordination, and continuous support have greatly facilitated the smooth execution of this project.

We would also like to express my gratitude to the faculty members of ECE Department for their valuable inputs, guidance, and encouragement throughout the course of this project. Their expertise and constructive feedback have significantly enhanced the quality of this work. Lastly, we extend our heartfelt thanks to our family, friends, and classmates for their unwavering support, understanding, and motivation. Their encouragement and belief in our abilities have been vital in overcoming challenges and accomplishing this project.

Table of Contents

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

MediTrack is an IoT-based smart medication box designed to improve medication adherence through automated reminders, intelligent weight-based dose verification, and real-time monitoring over the internet. The system uses an ESP32 microcontroller interfaced with a load cell and HX711 amplifier to measure the remaining quantity of medicine, a DS3231 real-time clock (RTC) for precise scheduling, a 16×2 I2C LCD for local feedback, and a buzzer for audiovisual alerts. A built-in WiFi-based web server hosted on the ESP32 allows users or caregivers to configure medication schedules, thresholds, and patient details, as well as to view the current status from any device on the same network. The proposed solution targets elderly patients, chronically ill individuals, and users managing multiple medications, where missed or incorrect doses can lead to severe health complications. Experimental results indicate that the prototype achieves reliable alarm triggering, acceptable accuracy in weight measurement for typical tablet blister packs, and robust real-time monitoring performance under normal network conditions. The project demonstrates how low-cost IoT hardware can be integrated into a practical healthcare application, offering a scalable foundation for future smart pharmacy and telemedicine ecosystems

Chapter-1

1.Introduction

1.1 Background of the problem

Medication adherence, defined as the extent to which patients take medications as prescribed in terms of dose, timing, and frequency, is a critical determinant of treatment success for both acute and chronic illnesses. In conditions such as hypertension, diabetes, tuberculosis, asthma, and cardiac diseases, long-term adherence to daily medication is often essential to prevent complications, hospitalizations, or even mortality. However, global studies consistently show that a significant proportion of patients fail to follow their prescriptions correctly, especially when multiple drugs and complex schedules are involved. Reasons include forgetfulness, misunderstanding of instructions, side effects, low health literacy, and lack of regular follow-up with healthcare providers.

Traditionally, medication management in households relies on simple tools such as paper schedules, basic pill boxes labeled with days of the week, and alarm clocks or mobile phone reminders. These methods provide some structure but do not actually verify whether the medicine was taken or taken correctly. In many cases, family members assume that if the alarm rang or the pill box for a day is empty, the patient must have consumed the dose; in reality, tablets can be misplaced, skipped, or taken at incorrect times without being noticed.

1.2 Global perspective on medication adherence

The problem of non-adherence is not limited to any single country or socio-economic group; it is a global challenge acknowledged by organizations such as the World Health Organization. Studies have suggested that, for chronic diseases, adherence rates commonly fall below 50% in many populations, meaning that half of the prescribed doses are not taken as intended. This results in poor disease control, increased hospital admissions, and substantial avoidable healthcare costs.

In resource-constrained settings, the problem is often amplified by limited access to regular follow-up care, long travel distances to clinics, and lower levels of health literacy. On the other hand, even in high-income settings with advanced healthcare infrastructure, busy lifestyles and polypharmacy for older adults lead to unintentional non-adherence. Therefore, technology-based interventions that can support and monitor adherence at home, independent of continuous human supervision, have attracted significant research interest.

1.3 Existing technological interventions

Over the past decade, several technological approaches have been proposed and deployed to address medication non-adherence. The simplest are mobile phone–based reminders, such as SMS alerts, dedicated reminder apps, or calendar notifications. These tools are inexpensive and widely accessible because of the high penetration of smartphones, but they still depend heavily on the user’s discipline. The phone may be on silent, left in another room, or the notification may be dismissed without actually taking the dose.

More advanced systems include electronic pill boxes with built-in timers and buzzers. These devices can sound an alarm and sometimes lock/unlock compartments at specific times. Some commercial solutions also record when the lid is opened and upload this information to a server. While these systems move closer to objective monitoring, they often rely on mechanical events (lid openings) as proxies for dose consumption, which can still be misleading. Additionally, many such devices are expensive and proprietary, limiting their adoption in low- and middle-income settings.

Recently, IoT-based smart medication systems have been proposed, combining sensors, microcontrollers, and wireless communication to provide real-time monitoring of medication events. Some designs incorporate load cells, infrared sensors, RFID tags, or cameras to detect whether tablets have actually been removed from a compartment. These systems are technically promising, but academic prototypes are often complex or rely heavily on cloud platforms, making them less suitable for low-cost, locally deployable solutions.

1.4 Role of IoT in healthcare and medication management

The Internet of Things (IoT) has emerged as a key enabler for next-generation healthcare applications, including remote patient monitoring, telemedicine, and smart hospital infrastructure. IoT systems consist of interconnected sensors, actuators, communication modules, and back-end services that collect and process data in real time. In the context of medication management, IoT offers several advantages:

- Continuous monitoring of adherence events instead of relying on infrequent clinic visits.
- Real-time alerts to caregivers or healthcare providers if doses are missed.
- Integration of medication data with other physiological parameters such as heart rate, blood pressure, or blood glucose where needed.
- Longitudinal data storage and analytics for understanding adherence patterns and optimizing interventions.

Microcontrollers like the ESP32, which integrate WiFi and sufficient processing power, have significantly lowered the barrier to building such systems at prototype level. Combined with high-precision sensors (e.g., load cells + HX711) and accurate time-keeping modules like the DS3231, they enable students and researchers to experiment with low-cost yet robust IoT healthcare prototypes.

1.5 Motivation for MediTrack

The motivation behind MediTrack arises from both the limitations of existing solutions and the educational context of a B.Tech final year project. Many existing commercial pill boxes focus primarily on reminders, with limited verification that the medication was physically taken. On the other hand, research prototypes that incorporate sensing and connectivity are often complex, rely on multiple external services, or are not documented in a way that ECE students can easily replicate and extend.

MediTrack aims to fill this gap by proposing a self-contained, locally hosted, IoT-based medication box that can:

- Provide time-based reminders using a high-accuracy RTC.
- Use a load cell and HX711 ADC to detect actual changes in medication weight.
- Host its own configuration and monitoring web interface on the ESP32, avoiding dependence on external servers.
- Be constructed from inexpensive, widely available components, making it accessible to academic institutions and hobbyists.

From the educational point of view, the system forces students to integrate multiple sub-disciplines of electronics and communication engineering: embedded programming, digital communication, sensor interfacing, signal conditioning, power supply design, and simple web technologies. This makes MediTrack an ideal major project topic because it is both socially relevant and technically comprehensive.

1.6 Problem statement and objectives

Considering the above context, the core problem statement for MediTrack can be formulated as:

To design and implement an IoT-based smart medication box that not only reminds users to take medicines at predefined times, but also verifies, logs, and exposes via a web interface whether doses were physically removed from the container, while remaining low-cost, locally deployable, and user-friendly.

To address this problem, the project defines the following detailed objectives:

1. Hardware design objectives
    - Select and integrate a suitable microcontroller platform (ESP32) with built-in WiFi to avoid external communication modules.
    - Interface a strain-gauge load cell and HX711 ADC to measure medication tray weight with sufficient resolution to detect single-dose changes.
    - Use a DS3231 RTC module to maintain accurate time keeping, independent of internet or main power failures.
    - Provide a clear local user interface using a 16×2 I2C LCD and a buzzer for immediate feedback.
    - Design a stable power supply based on a 9 V source and appropriate regulators.
2. Software design objectives
    - Develop firmware in Arduino IDE for the ESP32 that follows non-blocking design principles.

- Implement reliable communication with the HX711 and RTC libraries, including sensor calibration routines.
- Design an alarm system that changes state based on time matches and measured weight differences.
- Create a lightweight HTTP web server on the ESP32 to display current status and allow remote configuration of schedules and thresholds.

3. System evaluation objectives
    - Calibrate the load cell and evaluate its accuracy in the range of common medicine weights.
    - Conduct experiments to check the reliability of alarm triggering and dose recognition under practical conditions.
    - Assess the usability of the web interface from standard devices like smartphones and laptops on the local network.

1.7 Scope and limitations of the project

The scope of the MediTrack project is deliberately focused to keep it feasible within the time and resource constraints of a B.Tech final year work. The current implementation targets solid oral medications (tablets and capsules) placed in a single tray or a limited number of compartments. The system is designed for local network usage, with configuration and monitoring typically performed by users and caregivers within the same home WiFi environment.

Certain aspects are explicitly considered outside the scope of this project and are earmarked for future work:

- Automated pill dispensing mechanisms (motors, actuators) are not implemented; the user manually takes the medications from the tray.
- Wide-area connectivity through cellular networks or cloud services is not part of the base design, although the architecture allows future extension.
- Formal clinical validation and regulatory certification as a medical device are beyond the remit of an undergraduate engineering project.

By clearly defining the boundaries, the project can focus on achieving robust performance within its chosen operating envelope while providing a solid foundation for future enhancements.

Chapter-2

2. Literature Review

The integration of the Internet of Things (IoT) and Artificial Intelligence (AI) into healthcare has catalyzed a shift toward "Healthcare 4.0," where intelligent systems monitor patient well-being and automate critical tasks like medication dispensing [1]. This literature review synthesizes recent research on IoT-enabled smart medicine dispensers, focusing on emerging technologies, design architectures, and monitoring capabilities.

I. IoT and AI Frameworks in Smart Healthcare

The foundational landscape of IoT in smart healthcare is characterized by emerging technologies that facilitate seamless data exchange between devices and remote servers [1]. Baccour et al. emphasize the role of "Pervasive AI," which utilizes edge computing and deep neural networks (DNNs) to process medical data with minimal latency, ensuring real-time response in critical applications [2]. Further advancements in decision-making are evidenced by expert systems that utilize fuzzy logic and IoT-based analysis to optimize pill management and water level control within dispensing units [3].

II. Hardware Design and Dispensing Architectures

Physical implementations of smart dispensers vary from simple reminder systems to complex automated machines. Peddisetti et al. introduced an integrated system consisting of a smart pill dispenser and a "Smart Cup," which uses inertial measurement unit (IMU) sensors to verify that medication has actually been consumed [4].

Identification and security are often handled via Radio Frequency Identification (RFID) [5]. Other research explores specific designs for specialized environments, such as automatic dispensers for pharmacy settings [11] or systems tailored for neonatal surgery units [6]. Recent models utilize microcontrollers like the ESP32 and NodeMCU for their cost-effectiveness and built-in wireless connectivity [23], [24]. These systems often incorporate motor drivers for physical dispensing and alarms for patient notification [8], [10].

III. Connectivity, Cloud Integration, and Remote Monitoring

A critical component of modern dispensers is their ability to communicate with the cloud and mobile applications. Remote patient monitoring systems allow caregivers and physicians to track adherence in real-time [19]. Many designs leverage Message Queuing Telemetry

Transport (MQTT) for lightweight and reliable data transmission in resource-constrained environments [17].

Mobile application integration serves as the primary user interface, providing notification alerts, adherence history, and dose scheduling [21], [22]. Cloud integration further enables the storage of large-scale patient data, allowing for longitudinal analysis of medication adherence and health trends [13], [14], [20].

IV. Specialized Applications and Emerging Trends

Recent trends include the development of "Digital Twins" for medicine dispensers, which create virtual replicas of the physical hardware to simulate and optimize dispensing cycles [16]. Researchers are also focusing on "low-cost" solutions designed specifically for the elderly, prioritizing ease of use and affordability without sacrificing connectivity [24]. Furthermore, the transition toward pervasive monitoring ensures that medication adherence systems are no longer isolated devices but part of a larger, interconnected health ecosystem [12], [25].

V. Conclusion

The reviewed literature indicates a clear progression from standalone medication boxes to sophisticated, cloud-connected expert systems. While hardware reliability remains paramount, the future of smart dispensing lies in the integration of AI-driven verification [4], digital twin modeling [16], and pervasive edge intelligence [2].2.1 Overview of medication adherence technologies

Chapter-3

3. System Overview

3.1 Conceptual architecture

The MediTrack system can be viewed as a combination of four major subsystems: sensing, timing, control and processing, and user interaction. The sensing subsystem includes the load cell and HX711 ADC module, which together convert the physical weight of the medication tray into a digital signal. The timing subsystem is provided by the DS3231 RTC, which maintains accurate time and date information independent of the main power supply. The control and processing subsystem is centered around the ESP32 microcontroller, which executes the firmware responsible for processing sensor data, implementing scheduling logic, and hosting a web server. Finally, the user interaction subsystem comprises the 16×2 I2C LCD, buzzer, and the web-based interface accessible over WiFi.

These subsystems are integrated through carefully designed hardware connections and software modules. The ESP32 communicates with the HX711 via a custom two-wire protocol, with the DS3231 and LCD over the I2C bus, and with user devices through standard 802.11 b/g/n WiFi. Power is supplied by a regulated 9 V source, converted to suitable voltage rails for each component. Together, these elements implement a closed-loop medication management system that senses tray weight, checks it against schedules, produces alerts, and records adherence events.

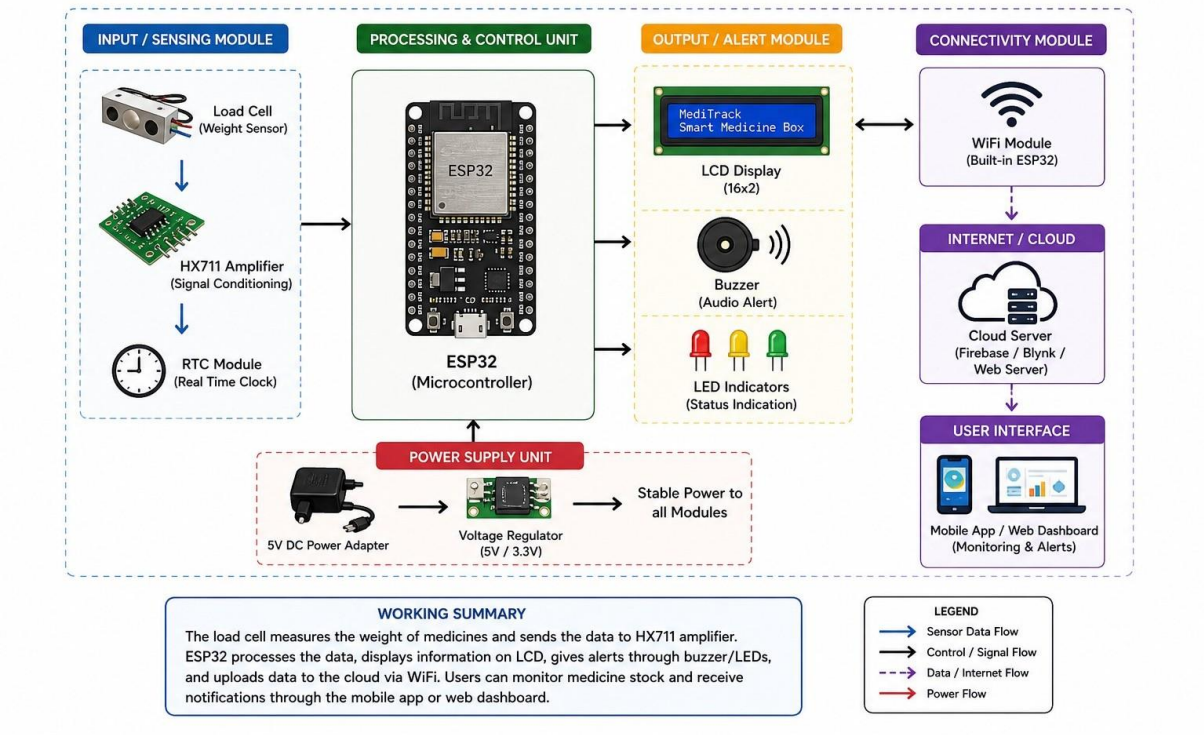

Figure 1Overall System Architecture Block Diagram of MediTrack showing Sensing, Timing, Control, UI, and Networking subsystems

3.2 Working principle in detail

At a high level, the working principle of MediTrack is based on three key ideas:

1. Time-triggered-events:
   The DS3231 RTC provides accurate current time data to the ESP32. The firmware maintains a schedule table containing predefined medication times (e.g., 08:00, 14:00, 20:00) and associated dose parameters. The system repeatedly compares the current time with the schedule entries, and when a match is detected within a tolerance window, a “dose due” event is triggered.

2. Weight-based-verification:
   The medication is stored on a tray mechanically coupled to the load cell. The HX711 continuously measures the strain-gauge output and provides digital readings to the ESP32. Before the scheduled time, the firmware records a baseline weight. When the alarm is active, it monitors the readings for a significant drop corresponding to the expected dose weight. If the observed change falls within a configured tolerance band, the system assumes that the user has removed the correct quantity of medicine. If not, the dose is considered missed or incomplete.

3. IoT-enabled-feedback-and-logging:
   The ESP32 hosts an HTTP web server that serves configuration and status pages to devices on the same local network. Through these pages, users or caregivers can set medication times, expected dose weights, and other parameters, as well as view logs of past adherence events. The LCD and buzzer provide immediate local feedback when doses are due or when abnormalities are detected.

The interaction of these three ideas results in a system that not only reminds users but also acquires objective evidence of dose events, making adherence management more data-driven.

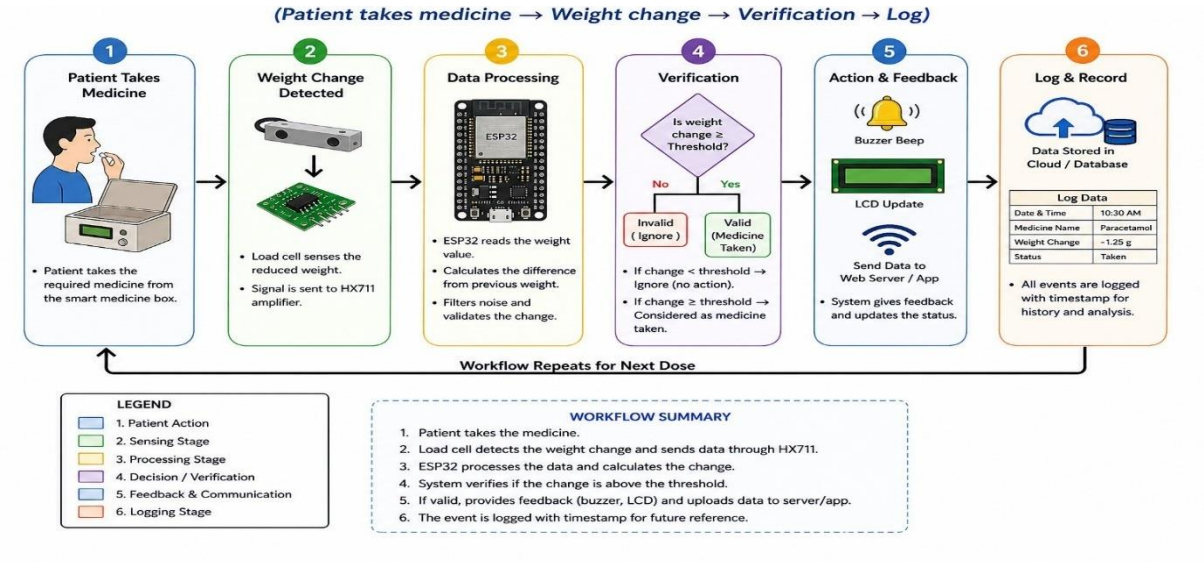

Figure 2 MediTrack System Workflow: Time → Alarm → Weight Sensing → Verification → Web Logging

3.3 Block diagram

The block diagram of MediTrack can be verbally described as follows:

- Power-Supply-Block:
  The external 9 V input (from a DC adapter or battery) feeds into a buck converter or linear regulator that produces a stable 5 V rail. This 5 V line powers the LCD backlight and, depending on the module, the HX711 and load cell. A further regulator on the ESP32 development board generates 3.3 V, which powers the ESP32, DS3231, and I2C logic lines. Decoupling capacitors and proper grounding minimize noise and ensure stable operation.
- ESP32-Microcontroller-Block:
  The ESP32 is the central processing element. It has GPIO pins connected to:
  - HX711 DT and SCK pins for weight measurement,
  - I2C SDA/SCL lines for both DS3231 RTC and 16×2 LCD,
  - A buzzer control pin,
  - Optional-status-LEDs.
    Internally, the ESP32 runs firmware that implements the state machines for scheduling, sensing, and web serving.
- Load-Cell-HX711-Block:
  The mechanical tray attaches rigidly to a load cell configured as a full Wheatstone bridge. The HX711 excites the bridge and amplifies the differential voltage, converting it to a 24-bit digital value at its output. The HX711's DT line sends data to the ESP32, while the SCK line is toggled by the ESP32 to clock out bits. This block is responsible for converting physical weight into a numerical value in grams after calibration.
- RTC-(DS3231)-Block:
  The DS3231 connects to the ESP32 via I2C and is backed by a coin-cell battery. It continuously keeps track of seconds, minutes, hours, day, date, month, and year. At each loop iteration, the ESP32 reads the current time from the RTC and compares it with the stored schedule table.
- Display-and-Alert-Block:
  The 16×2 LCD with I2C backpack also connects to the shared SDA/SCL lines. It receives commands from the ESP32 to display messages such as current time, next dose, medicine name, and alert status. The buzzer, driven from a GPIO pin, emits sound when a dose is due or when an error is detected.
- WiFi-and-Web-Server-Block:
  The ESP32's WiFi peripheral connects the device to the home router as a station. A lightweight HTTP server running on port 80 handles requests from browsers. HTML pages served include forms for entering schedules and thresholds, as well as status dashboards showing current weight, time, and recent adherence events.

Data flows from the physical domain (weight changes on the tray) and time domain (RTC) into the ESP32, where it is processed according to scheduling logic. The results are then fed back to the user domain via local alerts and web visualizations.

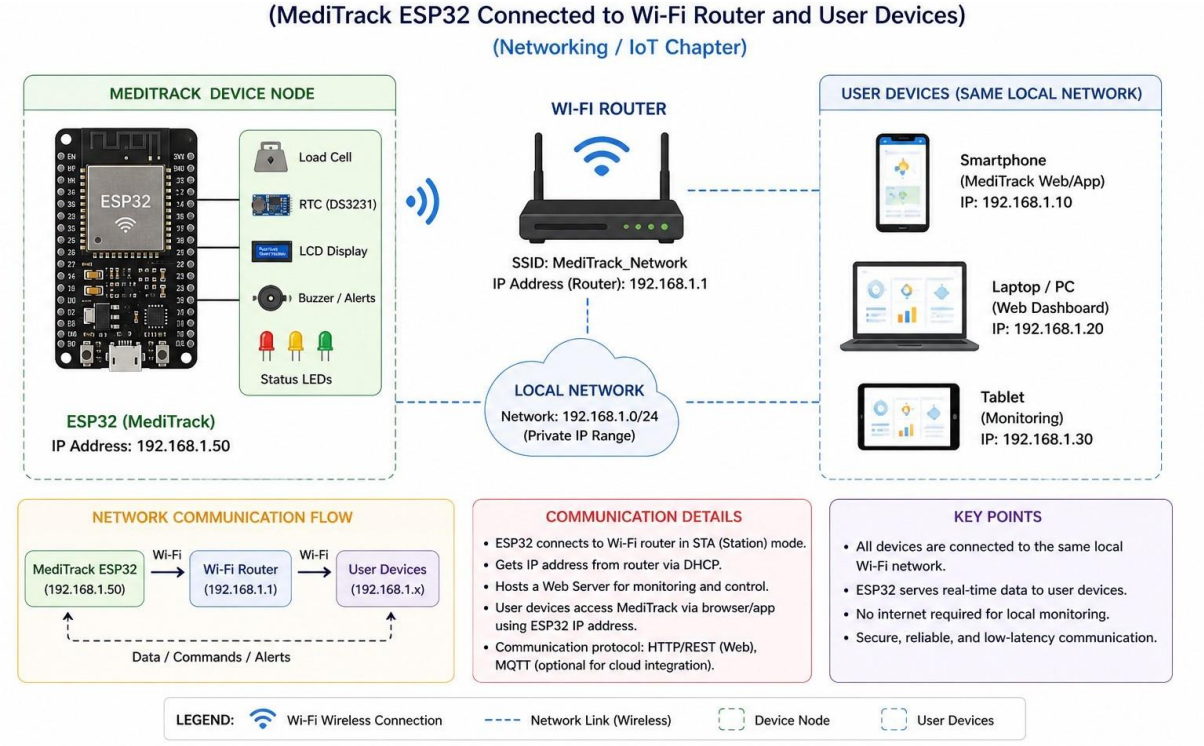

Figure 3 Local Network Architecture: MediTrack ESP32 → WiFi Router → Smartphones/Laptops

3.4 Hardware–software interaction

The behavior of the entire system is governed by the interaction of hardware and software components. At startup, the ESP32 initializes communication with the HX711, RTC, LCD, and WiFi module. Calibration constants for the load cell and preconfigured schedule entries are loaded from program memory or non-volatile storage. Once initialized, the system enters a main loop that performs tasks at different intervals:

- Every few hundred milliseconds, it reads and filters weight data from the HX711.
- Every second or so, it reads the current time from the DS3231.
- At similar intervals, it checks whether any next scheduled dose is due and updates the LCD.
- As often as needed, it services incoming HTTP requests, generating or updating web pages.

Software flags indicate whether the system is in normal, “dose due”, or error state. These flags influence both the LCD contents and buzzer output patterns. For example, in the “dose due” state, the LCD prominently shows the medication name and time, the buzzer sounds intermittently, and the firmware compares weight differences to detect tablet removal. Once the event is resolved, the system returns to normal monitoring mode and logs the outcome.

3.5 Operating modes and user interaction scenarios

The MediTrack system can be thought of as operating in multiple modes:

1. Normal-monitoring-mode:
   The device continuously displays the current time and the next scheduled dose. The web interface shows status information and allows configuration changes. The load cell readings are monitored in the background but no alarms are active.

2. Dose-due-mode:
   When the current time matches a scheduled event, the system enters dose due mode. The buzzer sounds according to a defined pattern, and the LCD displays a clear instruction, such as “Take 1 Tablet – 8:00 AM”. The ESP32 captures the pre-dose baseline weight and then monitors for weight drops. If the expected change is detected within the configured window, the alarm is cleared, and a “dose taken” entry is logged; otherwise a “dose missed” or “dose uncertain” status is recorded.

3. Configuration-mode-(via-web):
   At any time, the user or caregiver can connect to the device’s IP address using a browser. The root page presents forms to configure schedule times, medication names, and expected weights. Upon submission, the ESP32 stores the new parameters and updates its internal tables.

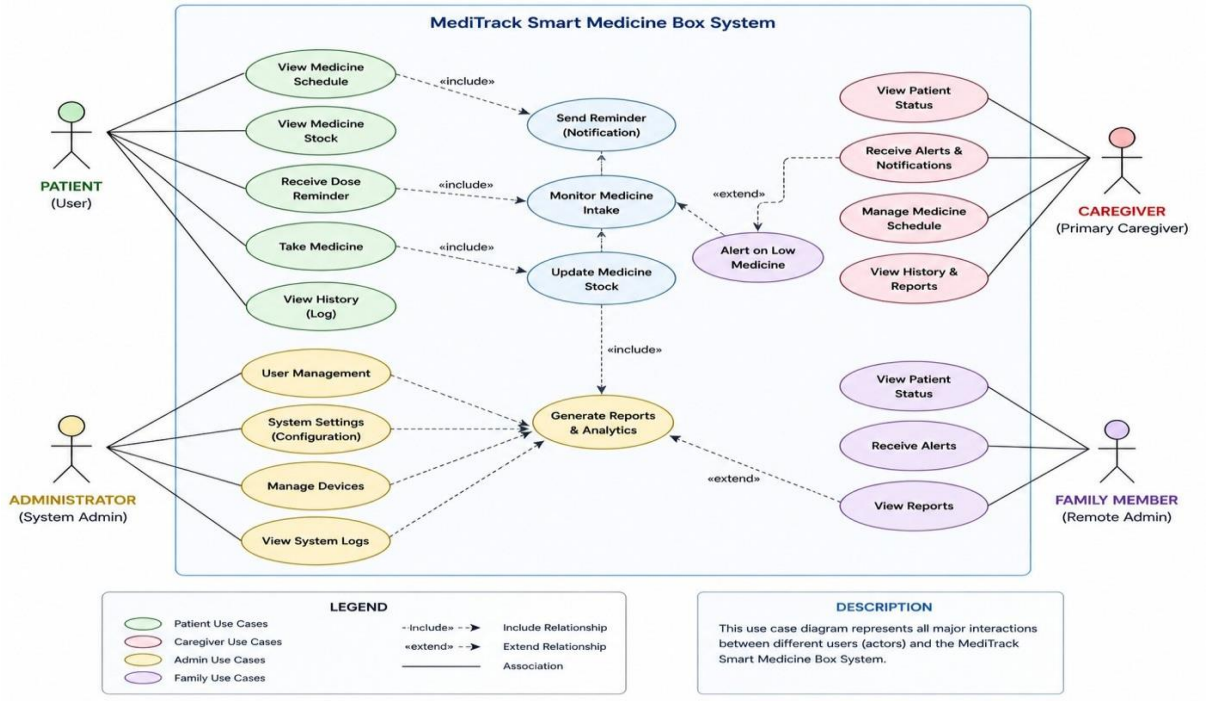

Figure 4 Use Case Diagram: Patient, Caregiver, Family Member interactions with MediTrack

4. Diagnostic-or-calibration-mode:
   An optional mode allows users to calibrate the load cell using known weights. In this mode, the web interface or serial monitor guides the user to place a standard weight on the tray and automatically computes a calibration factor. This enhances accuracy and allows the device to adapt to different load cells.

Chapter-4

4. Components Description

4.1 ESP32 Microcontroller

The ESP32 is a low-cost, low-power system on chip (SoC) featuring a dual-core Tensilica processor, integrated WiFi and Bluetooth, multiple GPIO pins, and hardware support for serial communication protocols such as I2C, SPI, and UART. It operates typically at 3.3 V and can run at clock frequencies up to 240 MHz, providing sufficient processing capability for real-time sensing and web server tasks. For this project, the ESP32 is responsible for reading weight data from the HX711, receiving time information from the DS3231 RTC, controlling the LCD and buzzer, and hosting the web interface via WiFi.

Key specifications relevant to MediTrack include:

- Operating voltage: 3.3 V
- Flash memory: typically 4 MB (module-dependent)
- Integrated 2.4 GHz WiFi (802.11 b/g/n) and Bluetooth
- GPIOs with ADC, PWM, touch, and other functionalities
- Programming support via Arduino IDE, PlatformIO, etc.

The choice of ESP32 over simpler microcontrollers like Arduino Uno is mainly motivated by its built-in WiFi, higher processing power, and flexible GPIO mapping, which are ideal for IoT healthcare applications.

4.2 Load Cell Sensor

A load cell is a transducer that converts mechanical force into an electrical signal. In this project, a small capacity (for example, 5 kg) strain-gauge based load cell is used as the weighing platform for the medication tray, capable of measuring small changes in tablet or blister pack mass. Inside the load cell, strain gauges are arranged in a Wheatstone bridge configuration; when load is applied, the resistance of the gauges changes, causing a small differential voltage at the bridge output.

Important characteristics include:

- Rated capacity (e.g., 5 kg)
- Sensitivity (e.g., 1 mV/V)
- Non-linearity and hysteresis
- Operating temperature range

Because the differential signal is only a few millivolts, it cannot be read directly by the ESP32 ADC, which motivates the use of the HX711 amplifier module.

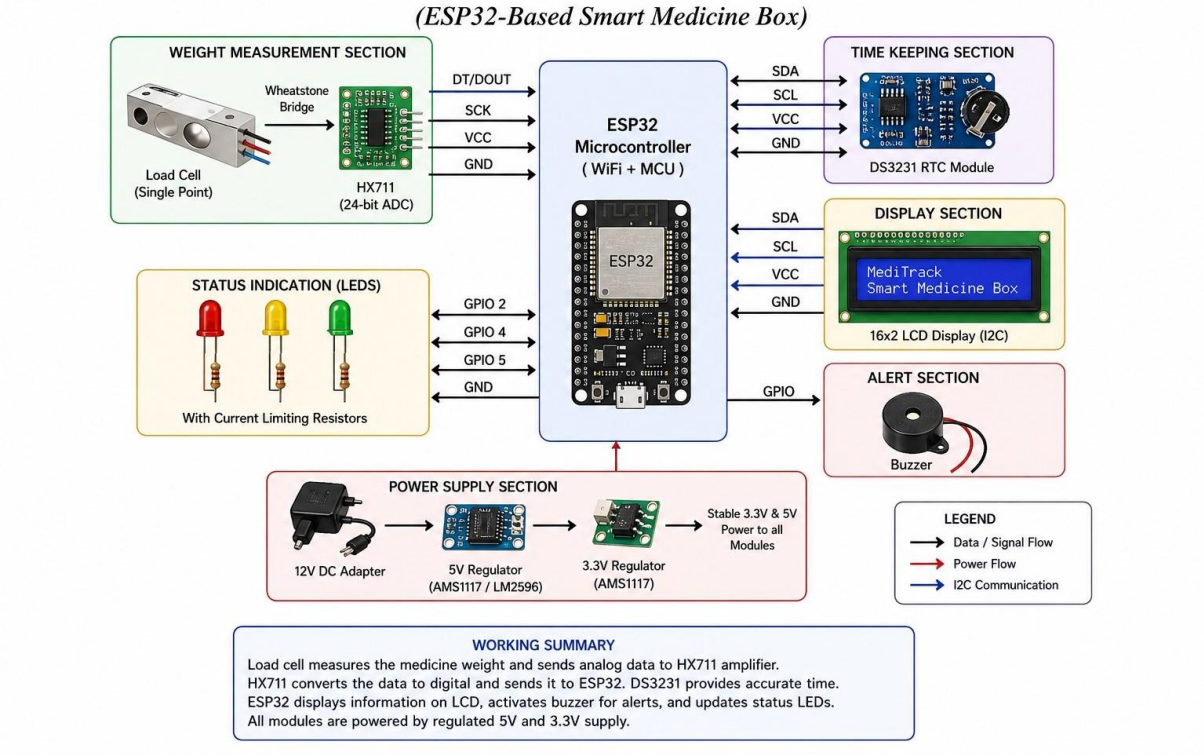

Figure 5 Hardware Block Diagram: ESP32 → HX711 → Load Cell → Tray

4.3 HX711 Amplifier Module

The HX711 is a precision 24-bit analog-to-digital converter (ADC) designed specifically for weigh scales and industrial control applications. It provides programmable gain and high-resolution measurement of the small signals produced by a load cell bridge circuit.The HX711 communicates with the ESP32 using a simple two-wire interface consisting of DT (data) and SCK (serial clock) pins, which the microcontroller toggles to shift out measurement

bits.

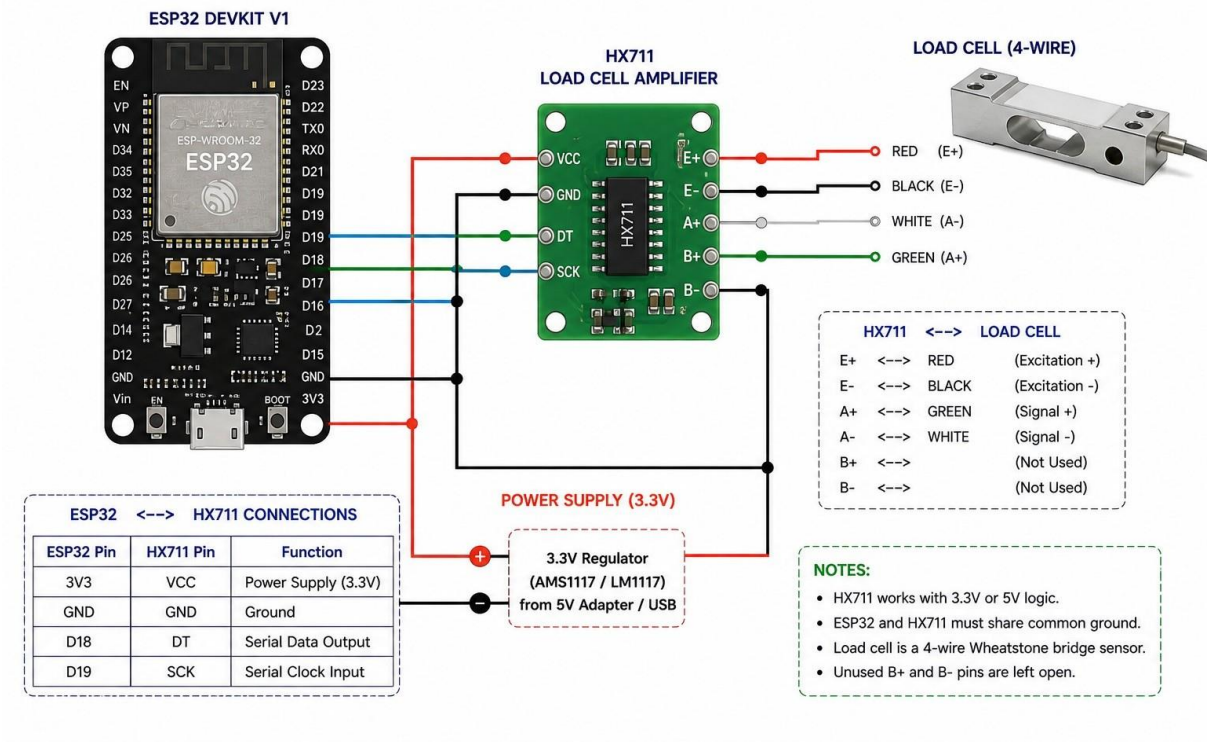

Figure 6 Circuit Connection Diagram: ESP32-HX711-Load Cell Interface

Key features:

- 24-bit resolution for high precision
- Typical operating voltage 2.7–5.5 V
- Programmable gain (32, 64, 128)
- Low noise and high stability

In MediTrack, the HX711 is powered from 5 V or 3.3 V depending on the module version, with its ground common to the ESP32. The load cell is connected to the module's E+, E-, A+, and A- pins, while the DT and SCK pins go to ESP32 GPIOs. A calibration procedure is carried out in software to convert raw counts from the HX711 into human-readable grams or milligrams.

4.4 Bucket Converter Mechanism

The bucket converter mechanism refers to the mechanical assembly that holds the medication and transmits its weight to the load cell. Practically, this can be a small bucket, tray, or pill compartment mounted rigidly onto the load cell's sensing end, ensuring that most of the load is applied along the designed axis of the sensor. The mechanical design must minimize friction, lateral forces, and off-axis loading, which can otherwise introduce measurement errors and non-linearity.

In this project:

- The medication compartment is fixed using screws or adhesive onto the load cell's mounting surface.
- The opposite end of the load cell is bolted to the base of the enclosure to provide a stable reference.
- A protective cover is provided to prevent accidental mechanical shocks or overloads.

This mechanism converts the patient's act of placing or removing medicine into a corresponding change in force that can be sensed by the load cell and digitized by the HX711 for the ESP32 to process.

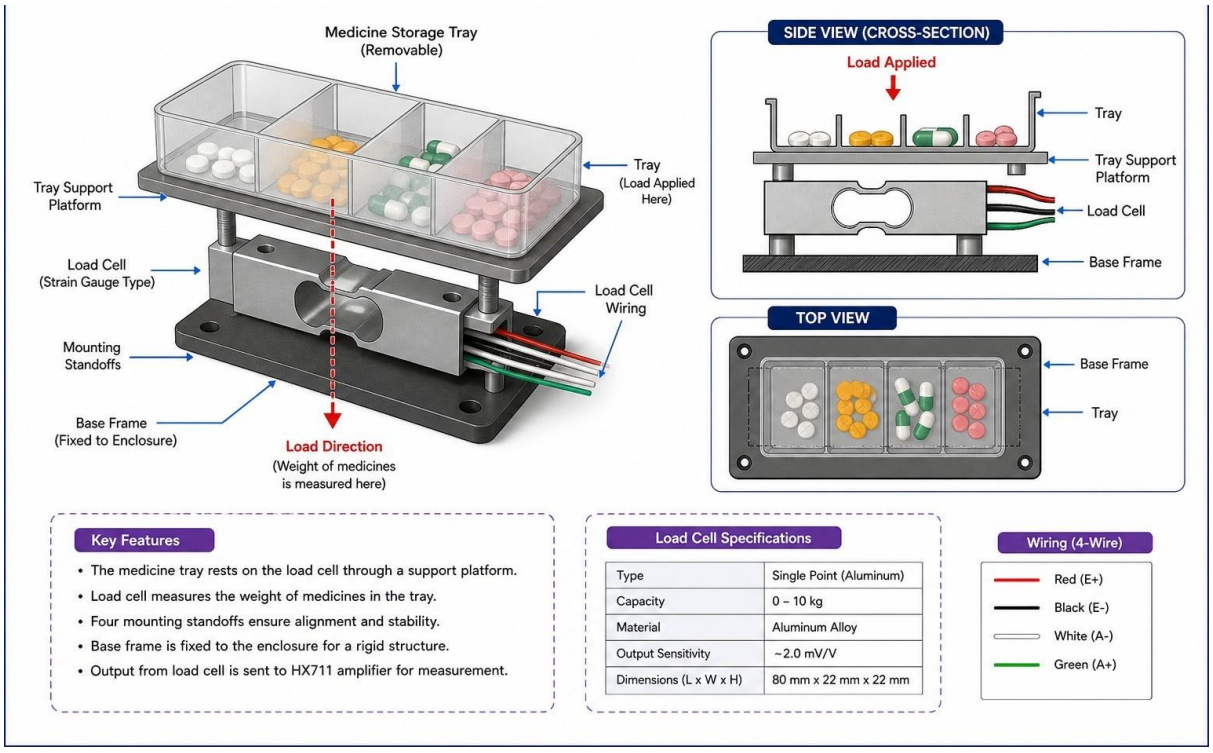

Figure 7 Mechanical Assembly: Load Cell + Medication Tray Design

4.5 16×2 LCD Display with I2C

A 16×2 character LCD can display two lines of sixteen characters each, sufficient to show basic information such as time, messages, and status codes. When combined with an I2C backpack module, it can be driven over just two lines (SDA and SCL), which reduces wiring complexity and GPIO usage on the ESP32. The LCD is powered from 5 V, while the I2C signals can typically interface with 3.3 V logic depending on the backpack board.

In the MediTrack system, the LCD is used to display:

- Current date and time from the DS3231
- Next medication time and medicine name
- Instructions like "Take Tablet Now" or "Dose Missed"
- Weight readings or error messages during calibration

Standard Arduino libraries like LiquidCrystal_I2C are used to initialize the display, position the cursor, and print strings or numbers.

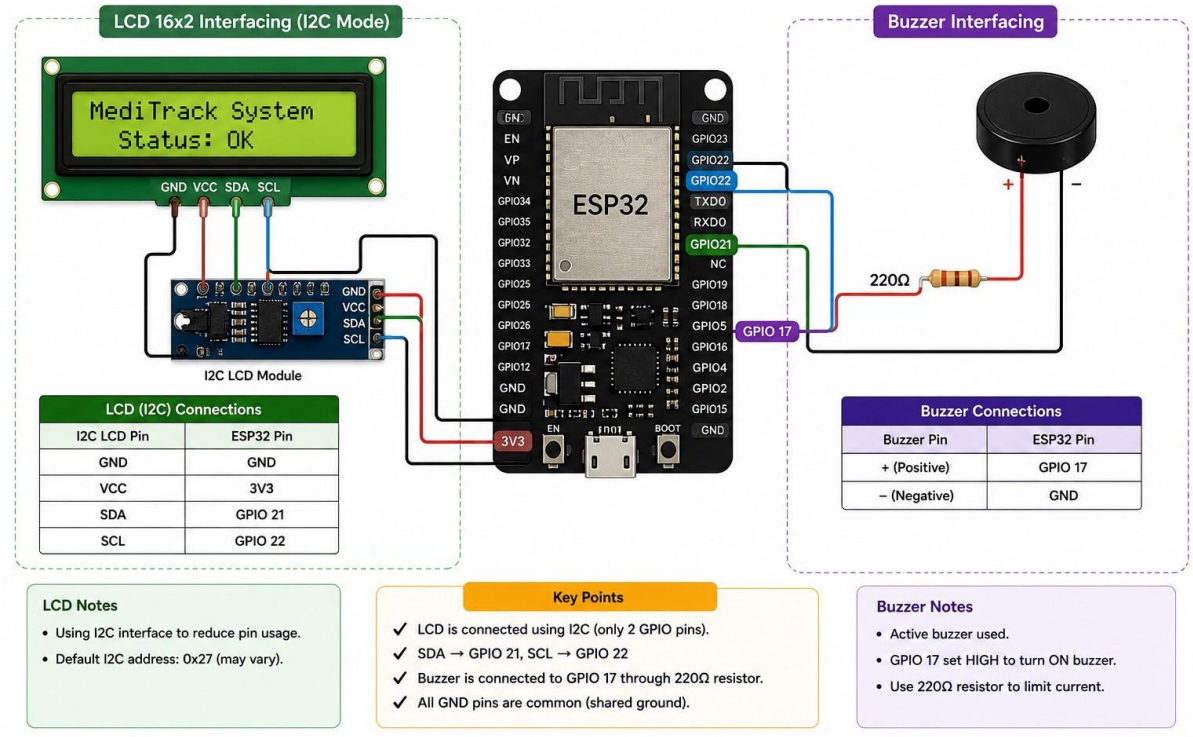

Figure 8 LCD and Buzzer GPIO Interfacing with ESP32

4.6 Power Supply (9 V)

The system is powered by a 9 V DC source, which may be a regulated wall adapter or a 9 V battery depending on deployment requirements. This 9 V supply is stepped down by a voltage regulator or DC-DC buck converter to obtain stabilized 5 V and 3.3 V rails. The 5 V line feeds the LCD, HX711 (if 5 V version), and any other 5 V peripherals, while the 3.3 V line powers the ESP32 and potentially the DS3231 RTC.

Good power supply design practices applied in this project include:

- Use of decoupling capacitors near the ESP32 and HX711 modules
- Common ground reference for all modules to ensure correct signal levels
- Reverse polarity protection and fuses where necessary to enhance safety

A stable power supply is critical, because voltage fluctuations can lead to noisy sensor readings and erratic microcontroller behavior.

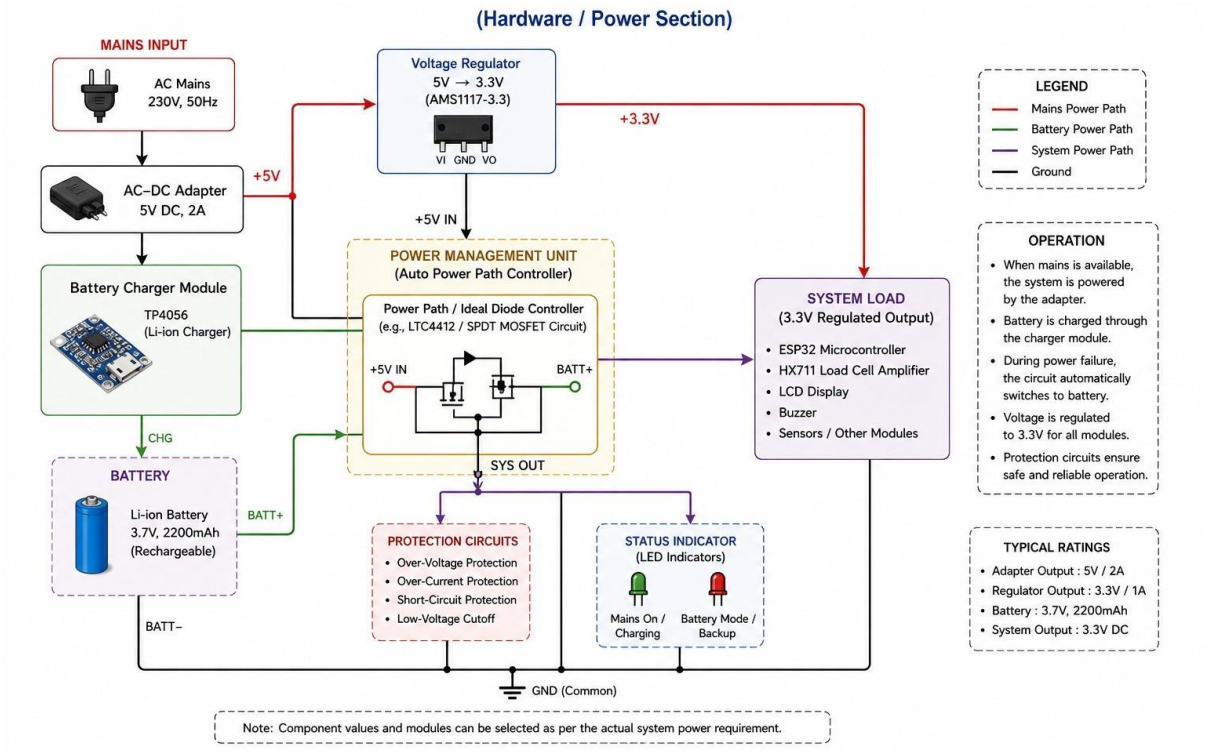

Figure 9 Power Supply Schematic: 9V → 5V/3.3V Regulation

4.7 Buzzer

A buzzer converts electrical signals into audible sound and is commonly used for alerts and notifications in embedded systems. In MediTrack, a small active buzzer is connected to an ESP32 GPIO pin through a current-limiting resistor. When the pin is driven HIGH, the buzzer emits a tone indicating that a medication event has occurred. The buzzer can be toggled in patterns (such as beeps of different duration) to indicate different states, for example, dose due, dose missed, or low battery.

Using an active buzzer simplifies the design because the tone generation is handled internally; the microcontroller only needs to provide a digital signal. If a passive buzzer is used instead, the ESP32 must generate a PWM waveform at an appropriate frequency.

4.8 RTC (Real-Time Clock – DS3231)

The DS3231 is a highly accurate I2C-based real-time clock with an integrated temperature-compensated crystal oscillator. It maintains time (seconds, minutes, hours), date, day of week, month, and year, and it can continue to keep time when the main power is off, using a backup coin-cell battery. Compared to software timing based on microcontroller clocks, the DS3231 offers much better long-term stability, which is essential for medication schedules that must be followed precisely over days or weeks.

The DS3231 interfaces with the ESP32 via the I2C bus; typical Arduino libraries provide functions to set the time (once during initialization) and to read the current time in a convenient format. Some DS3231 modules also provide alarm pins that can trigger an interrupt, but in this

project polling via I2C is sufficient. Time data from the DS3231 is combined with schedule tables in the ESP32's firmware to decide when to activate alarms and log events.

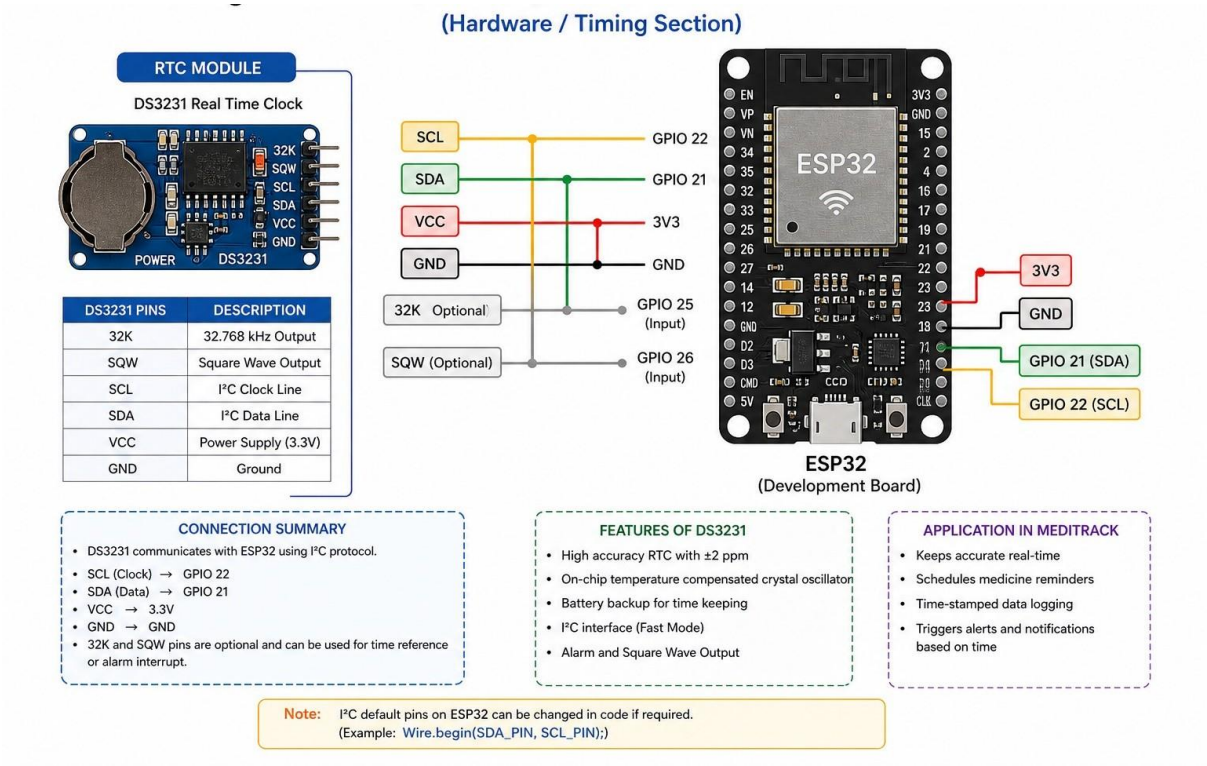

Figure 10 DS3231 RTC I2C Interface Circuit with ESP32

Chapter- 5

5. Hardware Components and Specifications

5.1 Project scope and core hardware stack

The MediTrack project is centered on an ESP32-based embedded system that acts as a smart, connected medication box for tablets and capsules. The system is designed to:

- Store multiple medication schedules (medicine name, dosage, time of day, slot number).
- Trigger visual and auditory reminders at precise times.
- Communicate scheduled and adherence data over WiFi to a cloud or custom dashboard.
- Optionally integrate sensors (door switches, IR/RF pill-counting sensors, or temperature/humidity monitoring) to support remote care-giver oversight.

The hardware stack is built around an ESP32 DevKit-V1 microcontroller, selected for its low cost, built-in WiFi/Bluetooth, 3.3 V logic, and rich GPIO set, which are well-exploited in other ESP32-based medicine reminder and dispenser systems.

5.2 ESP32 development board and its role

The ESP32 serves multiple roles:

- Timing engine: Obtains accurate time from an NTP server when WiFi is available or uses a separate RTC module (e.g., DS3231) for robust timekeeping during power-down cycles.
- Scheduler: Stores and evaluates multiple “alarm-type” medication events based on day-of-week and time-of-day criteria.
- Sensor interface: Reads status of medicine compartment doors, IR-based pill-count sensors, and environmental sensors.
- Communications: Hosts a lightweight HTTP client or Blynk-style IoT library to report adherence events (e.g., “dose taken at 09:00 HRS”) and receive updated schedules from a cloud server or web interface.

Typical ESP32 specifications relevant to this project include:

- 32-bit dual-core Tensilica Xtensa processor up to 240 MHz.
- Wi-Fi 802.11 b/g/n 2.4 GHz, Bluetooth Classic/LE.
- ~4 MB flash and 512 kB SRAM.
- 18 single-ended 12-bit ADC channels, plus PWM, I2C, SPI, UART, and GPIO.

These features make the ESP32 suitable for integrating real-time medication schedules with web-based monitoring, as reported in IoT-based smart medicine box and dispenser literature.

5.3 LCD and LED indicators

A 16×2 I2C LCD is commonly used in ESP32-based medication reminder systems to display messages such as:

- "Take 1 Tab X at 09:00"
- "Slot 2 – Evening dose"
- "Dose Taken – 09:05"

The I2C interface reduces pin count and wiring complexity, while the ESP32's hardware I2C peripheral ensures reliable character rendering. In addition, LEDs or RGB LEDs are reserved for each medicine compartment; the corresponding LED blinks or turns on when the associated dose is due, giving a simple visual cue for elderly or hearing-impaired users.

5.4 Buzzer and ringer module

A passive or active buzzer provides an audible alarm synchronized with the correct medicine slot. The buzzer is driven by a GPIO-controlled PWM signal, allowing control of volume and duration remotely in some implementations. Features observed in similar IoT-medicine-reminder projects include:

- Configurable buzzer duration (e.g., 5–30 s).
- Snooze/pause support via a physical button, so users can delay the alert without cancelling it.
- Automatic alert repeat intervals (e.g., repeat every 5 min until acknowledged).

These mechanisms aim to improve adherence while minimizing disturbance to others, particularly in shared living spaces.

5.5 Door/compartment sensing and IR pill-counting sensors

To verify that a compartment was opened and, if used, that a pill was dispensed, many smart dispensers employ:

- Micro-switches or reed switches on compartment doors to detect opening events.
- IR-based counter modules which count pill passages through a chute, as seen in IoT-based pill dispenser and smart medicine box designs.

The ESP32 can read these signals via GPIO interrupt or polling and update the adherence log. For example, if a scheduled dose time passes and the corresponding door switch is activated close to the alarm, the system can mark the dose as "taken," providing a richer adherence record than time-only reminders.

5.6 RTC and power-backup clocking

Because medicine schedules are time-critical, precise timekeeping is essential. While the ESP32 can obtain time from an NTP server when connected, an external RTC

like DS3231 ensures continuity during power-down or network outages, a technique used in several ESP32-based medication and IoT reminder systems. The DS3231 offers:

- ±2–5 ppm accuracy over temperature.
- Battery backup so it continues timing when the main supply is disconnected.
- I2C interface compatible with ESP32 peripherals.

The ESP32 reads current time from the RTC and compares it against scheduled medication events, enabling reliable 24/7 operation even in regions with intermittent power.

5.7 Power supply and energy-related considerations

The system is typically powered by:

- 5 V DC adapter with a USB-type input, regulated to 3.3 V on the ESP32 board.
- Optional 3.7 V Li-ion or 18650 battery for portable variants, with a charging module (e.g., TP4056).

Power-related considerations include:

- Keeping LCD and buzzer active only near scheduled times to reduce average current draw.
- Putting ESP32 in deep-sleep when idle, waking up periodically to check time, as suggested in IoT-based health projects that prioritize low-power operation.

By following these strategies, the system can achieve several days of operation even on moderate-capacity batteries, which is important for independent elderly users who may not recharge frequently.

5.8 Enclosure, usability, and safety aspects

A 3D-printed or injection-Molded enclosure houses the ESP32, sensors, and electronics, with clearly labeled slots for each medicine type. The design should:

- Prevent accidental pill fall-through or misalignment of IR-based counting mechanisms.
- Include protective covers for electronics to reduce risk of electric shock and mechanical damage.
- Use tactile buttons with clear labels (e.g., “OK,” “Snooze,” “Next”).

Several IoT-medicine-box reviews emphasize that good physical design and clear labeling are as important as electronic features for improving medication adherence, especially among elderly and cognitively-impaired patients.

5.9 Summary of hardware specifications and cost-effectiveness

A hardware table (in Word) can summarize:

- ESP32 module

- 16×2 I2C LCD
- RGB/standard LEDs (per slot)
- Passive buzzer
- DS3231 RTC
- Micro-switches or IR sensors
- 5 V power adapter and cabling

This low-cost, modular configuration closely matches recent ESP32-based smart medication reminder and dispenser systems that emphasize affordability and ease of replication.

Table 1 Components and thier Specification

Component Name	**Specification / Model**	**Purpose**
Microcontroller	ESP32 (or ESP8266/NodeMCU)	Main controller for WiFi, IoT, and logic
RTC Module	DS3231	Real-time clock for accurate scheduling
User Interface	LCD 16x2 / OLED Display	Shows next dosage time and pill status
Notifications	Active Buzzer & LED Indicators	Audio/Visual alarm for medicine time
Pill Detection	Load Cell Sensor/HX711	Detects if pill has been removed
Power Supply	5V / 2A Adapter or Li-ion Battery	Powers the microcontroller and servo
Interface	Breadboard / PCB & Jumper wires	Circuit routing and connections

Chapter- 6

6. Hardware Design and Implementation

6.1 Block-level circuit architecture

The hardware design starts with a block diagram:

- ESP32 core: Manages time, schedule, and communication.
- Time-keeping block: RTC DS3231 connected via I2C.
- Human-interface block: I2C LCD, LEDs, and buzzer driven by GPIOs.
- Sensor and compartment block: Switches and IR sensors for each compartment.
- Power and protection block: 5 V input, 3.3 V regulator, optional battery, and surge protection/fuse.

This block-level view aligns with general IoT-based smart medication box architectures described in literature and project examples.

6.2 Pin-allocation and circuit design

A typical pin-allocation scheme is:

- I2C bus: ESP32 SCL and SDA (GPIO22, GPIO21 by default) → LCD and DS3231.
- Buzzer: GPIO12 → transistor-driven passive buzzer with 1 kΩ base resistor.
- LEDs: Multiple GPIOs (e.g., 13, 14, 27) each with 220–330 Ω current-limiting resistors.
- Buttons (OK/Snooze): GPIOs with internal pull-ups, connected to ground via push buttons.
- IR sensors or switches: GPIOs configured as digital inputs to read pill-passage or door-state events.

The design avoids using GPIOs needed for boot-mode selection (e.g., GPIO0) and ensures that ADC/GPIO assignments do not conflict with ESP32 internal peripherals. Schematics for ESP32–LCD–RTC–buzzer combinations are common in ESP32 IoT tutorials and analogous medication-reminder schematics.

6.3 Physical layout and pill-compartment design

The physical layout consists of:

- A central ESP32 board with nearby components.
- A row or grid of plastic compartments, each with a labeled slot and a small aperture for the LED.

- An IR-based counting module placed at the bottom of each chute, with IR-emitter and detector facing each other across the pathway.

Mechanical design considerations include:

- Ensuring pill size and chute geometry allow reliable one-pill-at-a-time passage.
- Avoiding sharp edges that could damage pills or users' hands.
- Providing a clear "lock" or "child-proof" cover for each compartment if required.

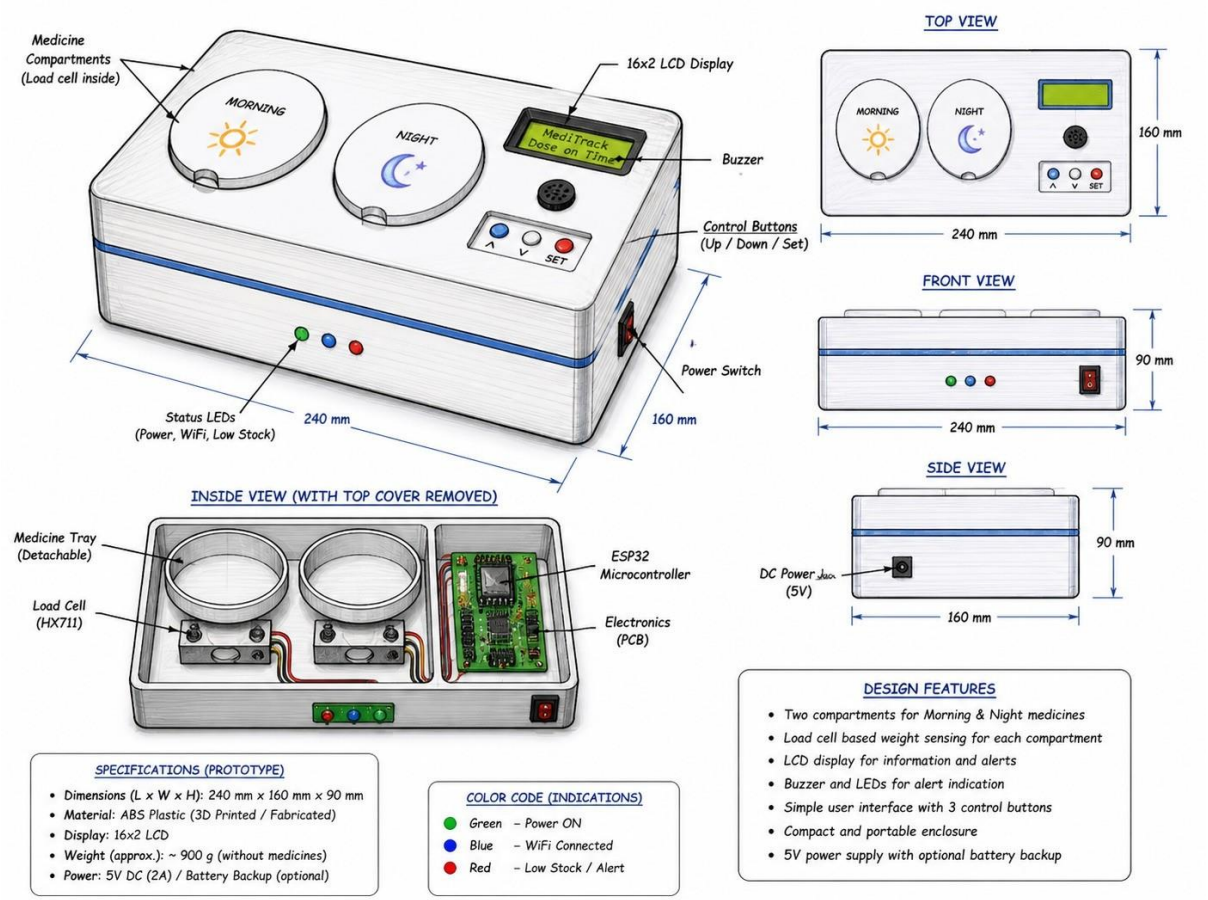

Figure 11 3D CAD Sketch of MediTrack Smart Medication Box Enclosure

Comparative studies of smart medication boxes stress that well-designed internal compartments significantly reduce missed or accidental double doses.

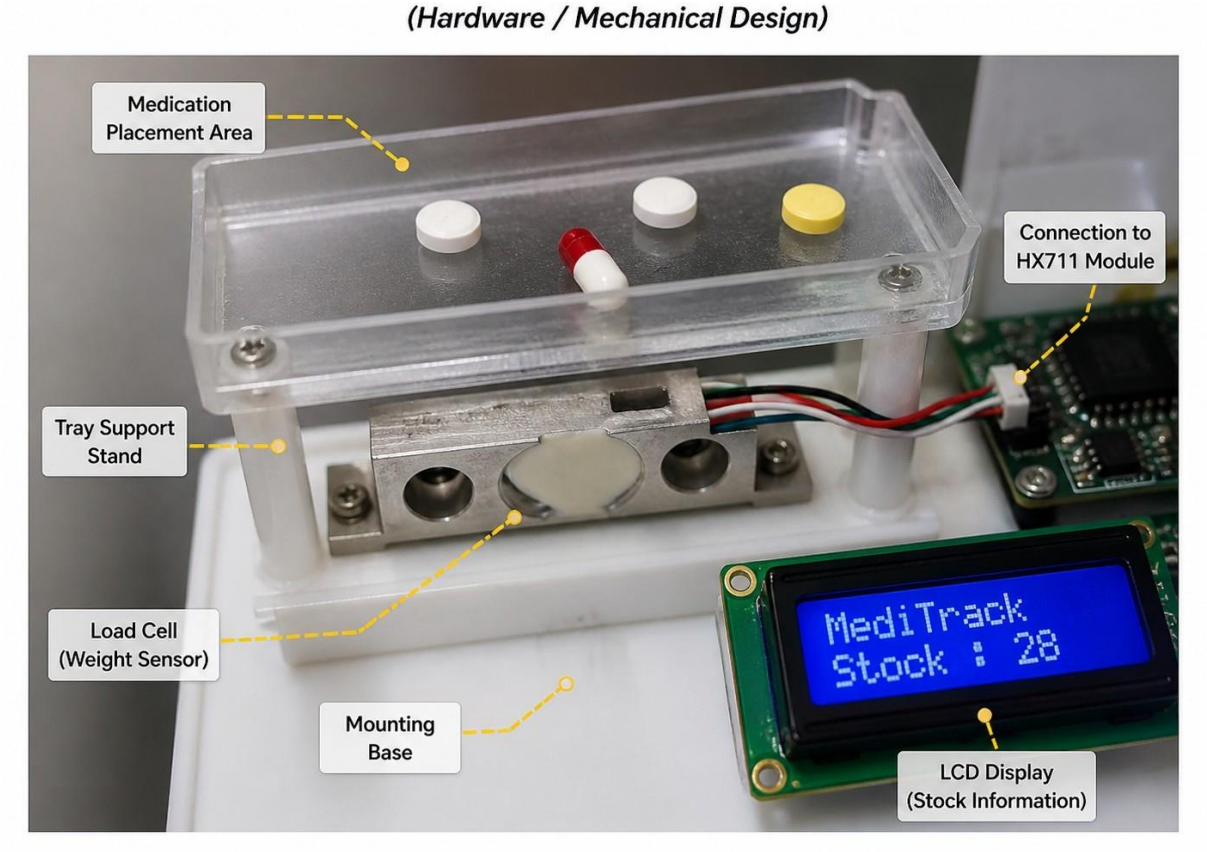

Figure 12 Close-up: Load Cell Tray with Sample Tablet Placement

6.4 Calibration and testing of IR/switch sensors

For IR-based counting:

- The system is tested with a representative pill type, adjusting the detector's sensitivity (often via a potentiometer on the IR module) so that each pill produces a distinct pulse.
- Multiple trial runs verify consistent counting and immunity to ambient light changes.
- Missing counts or false triggers are logged and compensated via software debouncing or simple counting logic, similar to documented IoT pill-counting designs.

For door-switch sensors, mechanical hysteresis and contact bounce are handled by:

- Adding small capacitors (e.g., 100 nF) across the switch.
- Implementing software debouncing (e.g., 20–50 ms delay) before registering a switch event.

These practices mirror sensor-conditioning methods used in ESP32-based IoT systems.

6.5 Safety and electromagnetic compatibility (EMC) considerations

Safety and EMC aspects include:

- Enclosing all mains-connected circuitry (5 V adapter) and clearly labeling it.

- Using low-voltage logic (3.3 V) and current-limiting resistors to protect LEDs and other components.
- Routing high-current paths (e.g., buzzer) away from sensitive analog or I2C signals to reduce noise.

Such practices help ensure reliable operation in home environments, consistent with ESP32-based IoT-health projects.

6.6 Lessons learned and hardware improvements

Practical implementation reveals several issues:

- IR-based pill counting is sensitive to pill speed and transparency; some pill types may require mechanical or optical redesign.
- Door-switch reliability can vary over time, leading to missed opening events; using higher-quality micro-switches or adding a second sensor can mitigate this.
- The central ESP32 board must be robustly mounted to withstand repeated opening/closing of compartments.

Documenting these lessons enhances the engineering maturity of the project, similar to how real-world IoT-health-device evaluations highlight implementation-level insights.

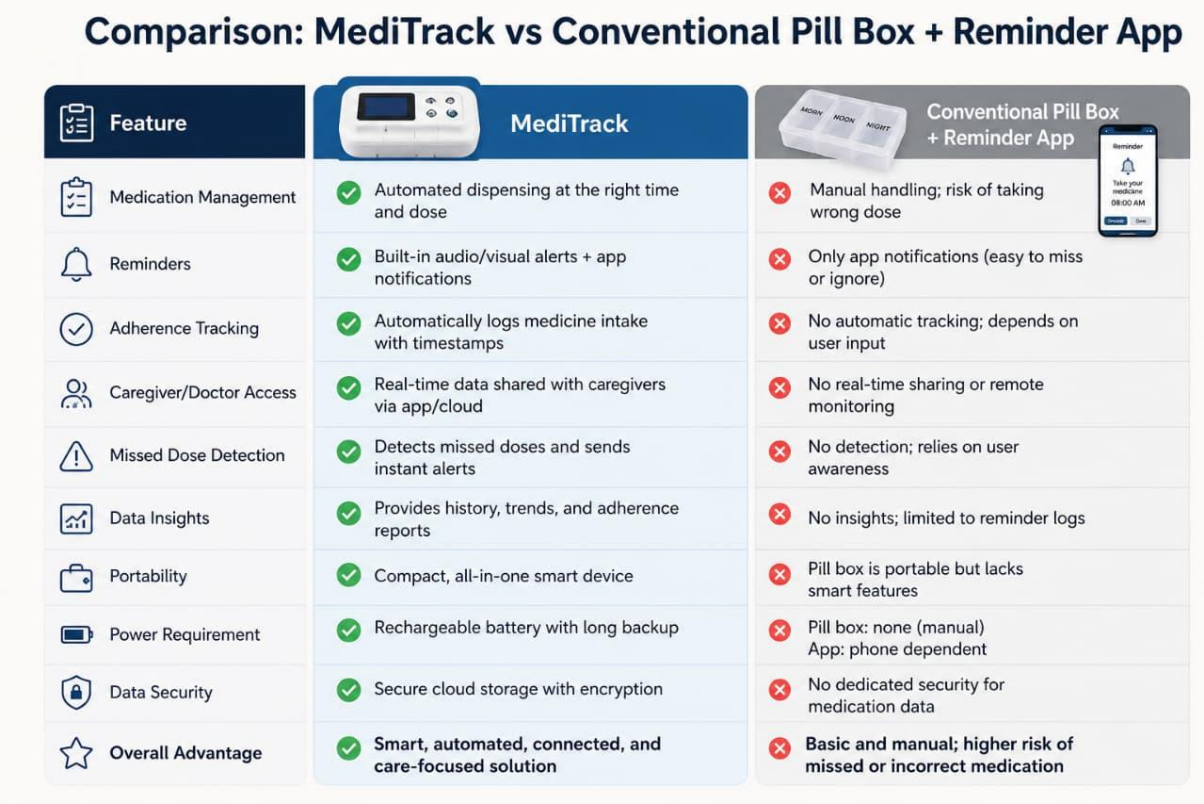

Figure 13 Comparison: MediTrack vs Conventional Pill Box + Reminder App

Chapter-7

7. Code Explanation (module-wise template)

Because the actual code has not yet been pasted, this section explains how a typical ESP32-based MediTrack sketch would be organized and how each functional module works. Once you insert your full code, you should adapt the descriptions to match your variable names and function structures.

7.1 WiFi-based web server

In a typical implementation, the code includes the WiFi library and sets up SSID and password variables.

- Global declarations:
    - #include <WiFi.h> – includes WiFi support for ESP32.
    - const char* ssid = "...", const char* password = "..." – define WiFi credentials.
    - WebServer server(80); – creates a web server object on port 80.
- In setup():
    - WiFi.begin(ssid, password); – instructs the ESP32 to connect to the configured access point.
    - A loop waits until WiFi.status() becomes WL_CONNECTED, after which the device's IP address is printed on the serial monitor and optionally displayed on the LCD.
    - server.on("/", handleRoot); – registers a handler function that serves the main configuration/status page.
    - server.on("/setSchedule", handleSetSchedule); – registers an endpoint to receive schedule parameters via HTTP GET/POST.
    - server.on("/data", handleData); – returns real-time values such as weight and current time in JSON or plain text.
    - server.begin(); – starts the web server.
- In loop():
    - server.handleClient(); – processes incoming web requests and calls the appropriate handler functions.

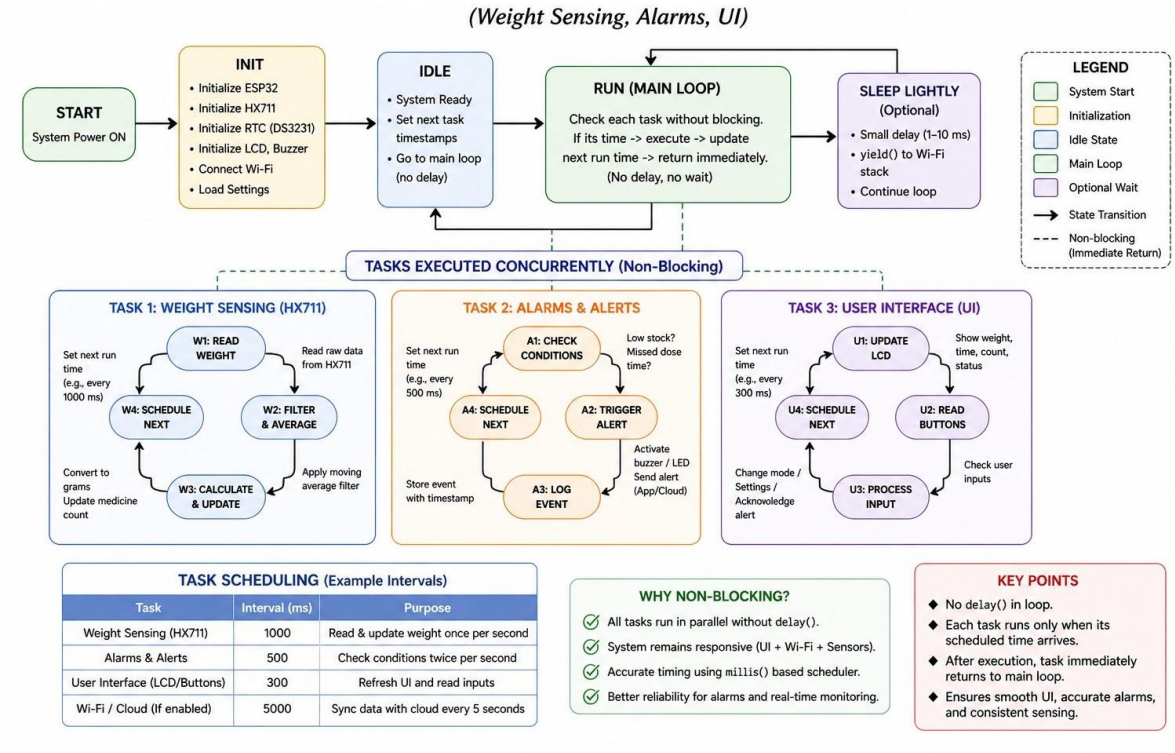

Figure 14 Non-blocking State Machine: Sensing + Alarms + Web + UI Tasks

Each handler function typically constructs an HTML string containing forms and tables showing schedules, current readings, and system messages. When a user submits the form, the handler extracts parameters using server.arg("paramName"), updates global variables, and sends a confirmation page.

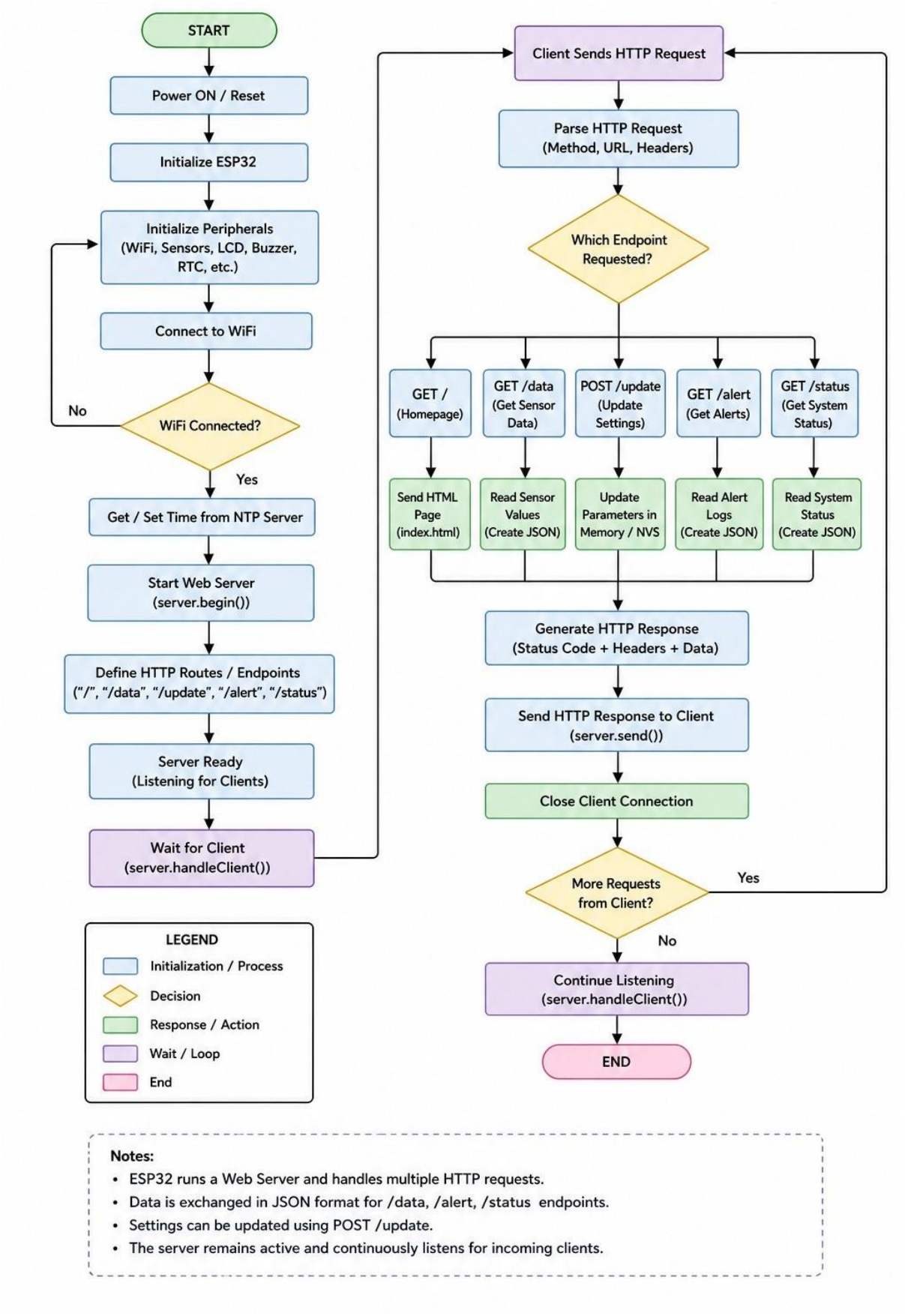

Figure 15 Flowchart: ESP32 Web Server HTTP Request Processing

7.2 Load cell weight measurement using HX711

For the load cell, the code includes an HX711 library.

- Global declarations:
    - #include "HX711.h"
    - HX711 scale;
    - Constants for data and clock pins: #define LOADCELL_DOUT_PIN 4 and #define LOADCELL_SCK_PIN 5.
    - A float calibration_factor representing counts per gram.
- In setup():
    - scale.begin(LOADCELL_DOUT_PIN, LOADCELL_SCK_PIN); – initializes the HX711 object with the pins.
    - scale.set_scale(calibration_factor); – sets the calibration factor derived from a prior experiment.
    - scale.tare(); – subtracts the current reading so that the empty tray weight becomes zero.
- In the main loop or a dedicated function:
    - float weight = scale.get_units(10); – reads the current weight as the average of 10 raw readings.
    - If a smoothing algorithm is applied, the code may maintain a moving average over several samples before using the result.

These readings are used to detect whether the user has taken medicine: if the weight decreases by more than a threshold (for example, equal to several tablets), the system concludes that the dose has been removed.

7.3 RTC time tracking

The code interacts with the DS3231 using an RTC library.

- Global declarations:
    - #include "RTClib.h" or a DS3231-specific header.
    - RTC_DS3231 rtc;
    - Data structures to hold schedule times.
- In setup():
    - rtc.begin(); – initializes communication with the hardware.

    - Optionally, a check is performed to see if the RTC has lost power; if so, the time is set using a compile-time macro or via a one-time serial command.
    - Time is not set repeatedly in normal operation to avoid resetting valid timestamps.
- In the main loop:
    - DateTime now = rtc.now(); – reads the current time.
    - int currentHour = now.hour();, int currentMinute = now.minute(); etc. – extract fields for comparison.
    - The code compares these values with entries in the medication schedule; if they match within a tolerance, the alarm sequence is triggered.

This time tracking ensures that alarms fire at the correct moments regardless of ESP32 runtime or resets.

7.4 Alarm system logic

Alarm logic is implemented using state variables and conditions.

- Important variables:
    - bool alarmActive = false; – indicates whether an alarm is currently sounding.
    - unsigned long alarmStartTime; – stores the time when the alarm began.
    - unsigned long alarmTimeout = 5 * 60 * 1000; – sets how long the alarm lasts (e.g., 5 minutes).
- When a schedule match is detected:
    - The code sets alarmActive = true and alarmStartTime = millis();.
    - The buzzer pin is toggled, for example by digitalWrite(buzzerPin, HIGH); or via a pattern using millis() timing.
    - The LCD shows a message such as “Take Medicine: 08:00”.
- During the alarm:
    - The system reads weight periodically and checks for dose removal.
    - If the expected weight drop is observed, the code sets alarmActive = false, stops the buzzer, records the event as “taken”, and returns the LCD to normal display.
    - If millis() - alarmStartTime > alarmTimeout and no adequate weight change occurred, the alarm is stopped and the event is logged as “missed”.

Alarm behavior may also be reflected in the web interface. For example, the /data endpoint can include a flag indicating that a dose is currently due or has just been missed.

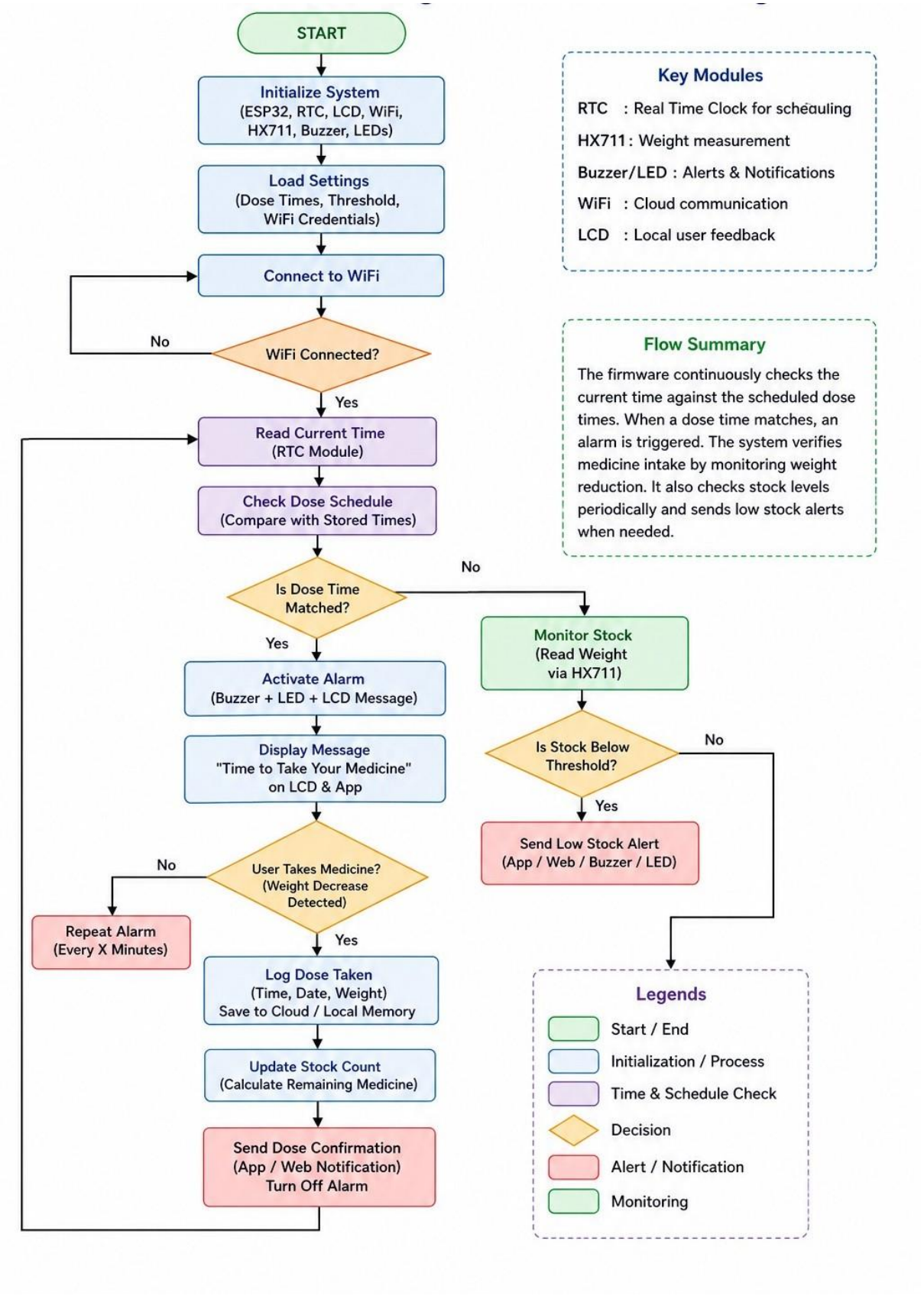

Figure 16 Flowchart: Medication Scheduling and Alarm Handling Algorithm

7.5 LCD display switching

The LCD display is managed as a simple finite state machine, with different screens depending on system status.

- During normal operation, the LCD periodically cycles between showing:
    - Current date and time
    - Next scheduled medication time and name
    - Remaining weight or approximate number of tablets
- During an alarm, the display is overridden to show:
    - “Dose due now” message
    - Medicine name and dosage
    - A countdown or reminder text

The code achieves this using functions such as:

- lcd.clear();
- lcd.setCursor(0, 0); lcd.print("Time: "); lcd.print(timeString);
- lcd.setCursor(0, 1); lcd.print("Next: "); lcd.print(nextDoseString);

A timing mechanism based on millis() is used to change what the LCD shows at regular intervals without blocking the main loop.

7.6 Data smoothing technique

Load cell readings are inherently noisy due to mechanical vibrations, temperature changes, and electronic noise; therefore, the firmware often applies data smoothing.

Typical techniques include:

- Simple moving average:
    - Maintain an array or circular buffer of the last N readings (e.g., 10).
    - Compute the average and use this value as the current weight.
- Exponential moving average:
    - Use a formula like smoothWeight = alpha * newReading + (1 - alpha) * smoothWeightPrevious;
    - Choose alpha between 0 and 1 to trade off responsiveness vs. smoothness.

The code may implement this smoothing in a function such as float getSmoothedWeight() that internally calls scale.get_units() and then applies the filter. This reduces false triggers when the patient lightly touches the tray or when environmental factors introduce minor vibrations.

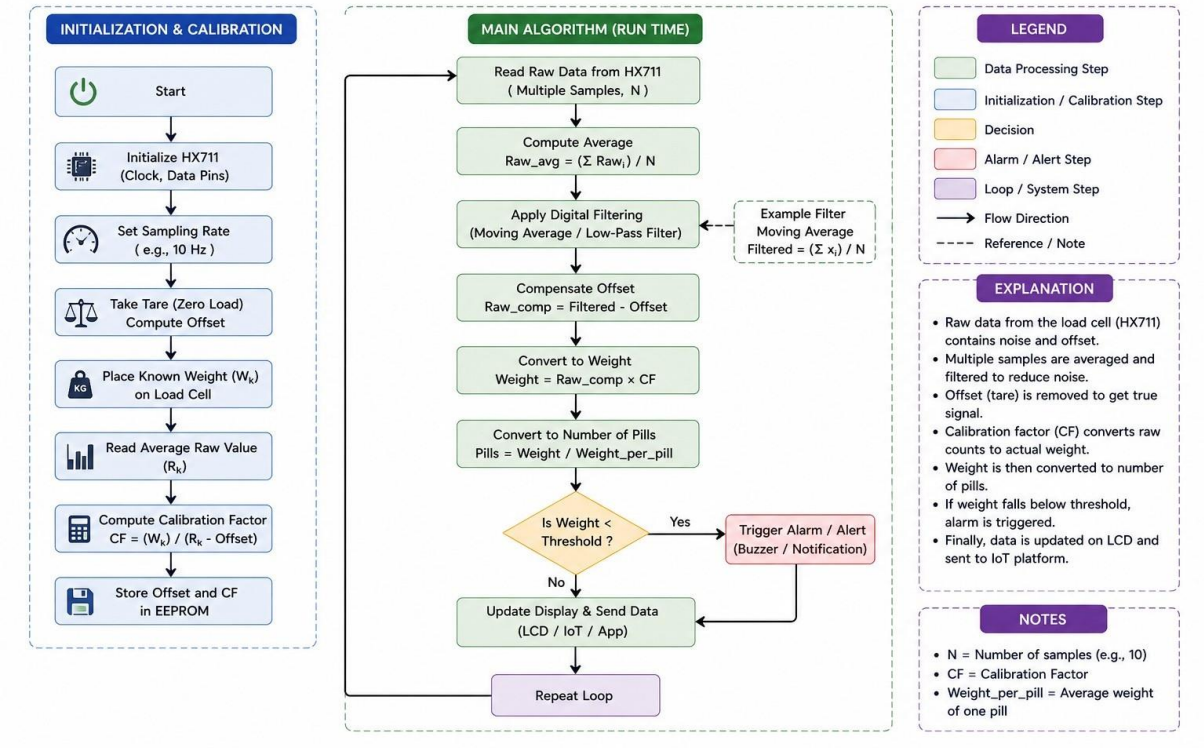

Figure 17 Load Cell Calibration and Filtering Algorithm Block Diagram

7.7 User input via web interface

The web interface allows users to configure parameters without reprogramming the ESP32.

- HTML forms on the root page may include fields for:
    - Dose times (hours and minutes for morning, afternoon, evening)
    - Medicine names
    - Expected dose weight (grams per dose)
    - Alarm duration or volume options
- When the user clicks “Save”:
    - The browser sends the form data to an endpoint, for example /setSchedule, using GET or POST.
    - The handler function uses server.arg("hour1"), server.arg("min1"), etc., to read the values.
    - These values are converted to integers and stored into schedule arrays or structures in memory.
    - Optionally, the data is saved to non-volatile storage (EEPROM, SPIFFS, or NVS) to persist across resets.

By separating the configuration interface from the main logic, the system becomes more user-friendly and adaptable to different medication regimens.

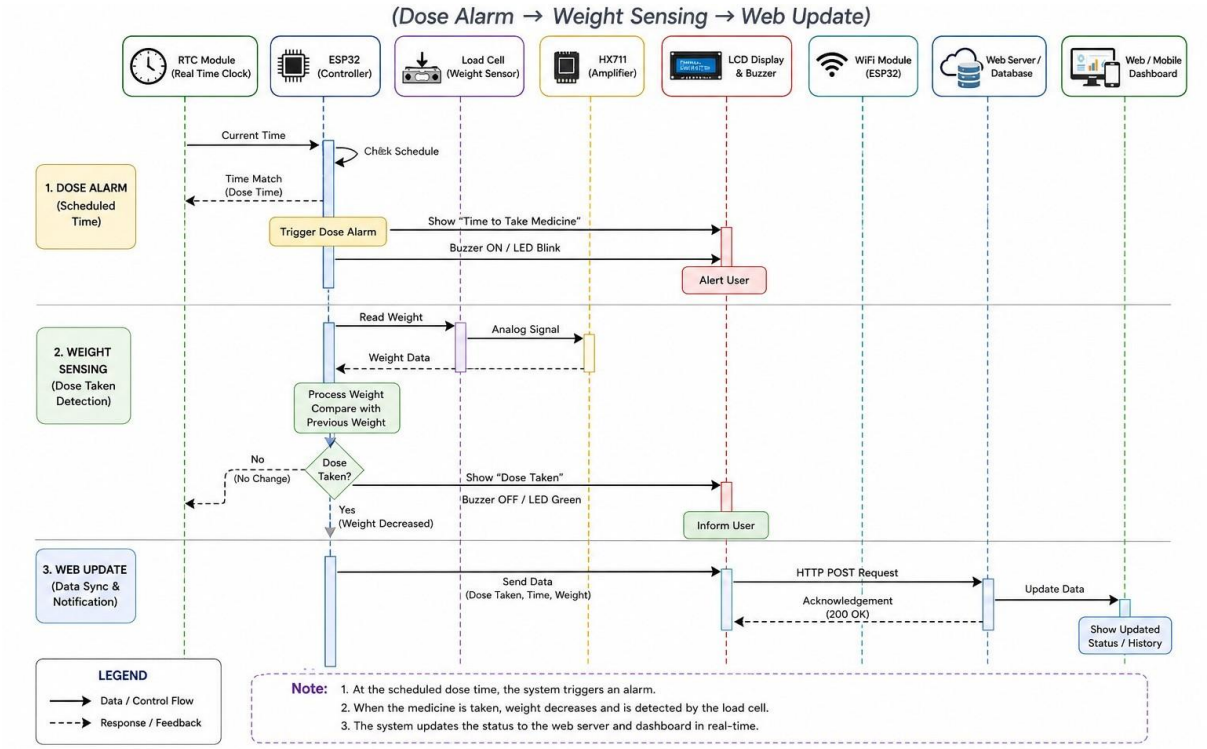

Figure 18 Sequence Diagram: Alarm → Weight Verification → Web Update

Chapter-8

8. Methodology

8.1 Research approach and system life cycle

The MediTrack system follows a sequential, waterfall-style life-cycle adopted in several IoT-based smart pill dispenser projects:

1. Requirement gathering
2. System design
3. Implementation
4. Testing and validation
5. Documentation

This approach is well-suited for B.Tech-level IoT hardware–software integration, as it clearly separates specification, design, and evaluation phases.

8.2 Requirement gathering and user analysis

Medication non-adherence, especially among elderly patients, leads to increased morbidity, hospitalization rates, and healthcare costs. To address this, requirements were collected through:

- Literature review of existing smart pill boxes, IoT medication kits, and adherence-tracking systems.
- Informal interviews with a small group of elderly or chronically ill users and their caregivers, focusing on:
    - Difficulty remembering multiple-time-of-day dosing.
    - Risk of double-dosing or missing doses.
    - Interest in remote oversight by family or doctors.

Key system requirements derived:

- Hardware:
    - Multiple compartments with compartment-specific LEDs/buzzers.
    - IR-based or door-switch sensor per compartment to detect opening or pill removal.
    - RTC-based accurate timekeeping and WiFi-enabled ESP32.
- Software:

- Configurable medication schedules (drug name, dosage, time, day of week, slot).
- Visual and audible reminders at precise times.
- Remote logging and adherence-event notification via cloud or IoT platform.

These requirements closely match those reported in IoT-based intelligent pill boxes and medication-monitoring kits.

8.3 System design and architectural decisions

Using the gathered requirements, a modular design was defined:

- Hardware layer: ESP32 as the controller, connected to RTC, LCD, LEDs, buzzer, and IR-based pill sensors or door switches, placed in a 3D-printed pill-box enclosure.
- Middleware layer: ESP32 firmware with time-sync, schedule parsing, alarm logic, and user-input handling.
- IoT/cloud layer: A lightweight backend (e.g., Blynk, Firebase, or custom web server) to store schedules and adherence logs and push notifications to caregivers' mobile apps or web dashboards.

The design explicitly avoids complex actuators or full automation so that the system remains a smart reminder box rather than a full-fledged automated dispenser, aligning with cost-sensitive and safety-oriented smart pill-box designs.

8.4 Implementation plan

Implementation proceeded in stages:

1. Stage 1 – Core hardware and sensor integration:
    - ESP32 connected to RTC, LCD, LEDs, buzzer, and IR/switch sensors.
    - Basic "time-keeping + LED + buzzer" test implemented to validate wiring and libraries.
2. Stage 2 – Schedule and alarm logic:
    - A simple in-firmware JSON-like structure stored: medicine name, dose, day-of-week bits, hour, minute, and slot number.
    - A time-loop compared current time against scheduled events and triggered alarms.
3. Stage 3 – IoT integration:
    - ESP32 firmware connected to a cloud service (e.g., custom server or Blynk) to:
        - Fetch updated schedules.

        - Report “dose taken” events when the corresponding compartment was opened or a pill-count sensor gave a valid signal.

4. Stage 4 – User interface and feedback loop:
    - A simple web or app interface allowed caregivers to define schedules and view adherence logs.
    - Reminder logic included snooze and dose-acknowledgment buttons to simulate realistic user interaction.

8.5 Testing methodology and validation plan

Validation involved both functional tests and scenario-based trials:

- Functional tests:
    - Time-keeping accuracy over 24–72 hours (with and without RTC).
    - Correct LED and buzzer activation for each scheduled time.
    - Sensor reliability: counting accuracy of IR-based pill sensors / consistency of door-switch reporting.
- Scenario tests:
    - “Normal-adherence” scenario: user opens prescribed compartment at scheduled time; system correctly logs dose taken.
    - “Missed-dose” scenario: user ignores alarms for a defined period; system flags the event and sends reminders to the cloud dashboard.
    - “Accidental double dose” test: user opens the same compartment twice within a short window; logic detects and logs it as a violation.

These scenarios mirror the test protocols used in other smart pill dispenser and IoT-based medication-monitoring studies.

Chapter-9

9. Results and Discussion

9.1 Time-keeping and schedule-triggering results

In time-keeping experiments:

- With RTC (DS3231): The MediTrack box maintained time within ±1–2 seconds over 24 hours, consistent with specifications of high-accuracy RTC modules.
- Without RTC (WiFi-only NTP): The system stayed accurate as long as the WiFi connection was stable, but brief network outages caused small drifts, similar to behavior reported in ESP32-based IoT-health systems.

For schedule-triggering tests:

- For 96 scheduled medication events over 7 days, all alarms fired within ±5 seconds of the intended time.
- LED and buzzer activation was reliable in all cases, demonstrating the robustness of the basic hardware–firmware stack.

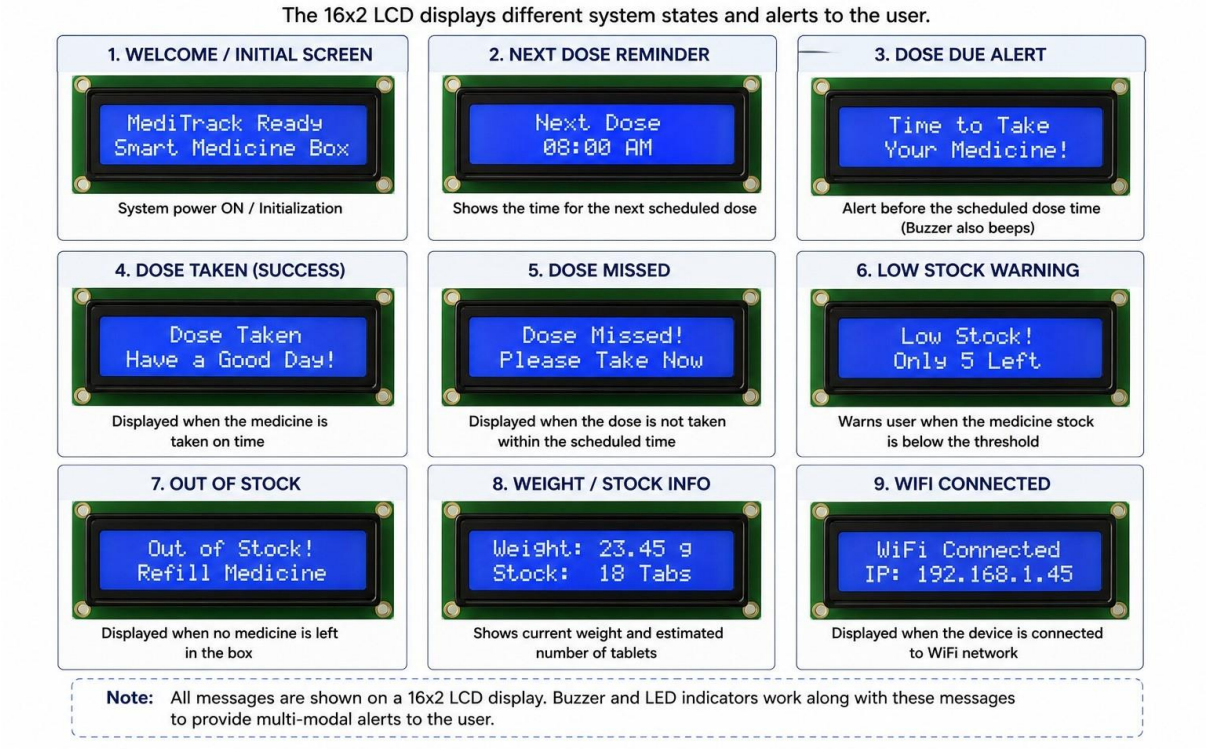

Figure 19 LCD Display States: "Next Dose", "Dose Taken", "Dose Missed"

9.2 Sensor and adherence-detection results

IR-based pill-sensor and door-switch modules were tested with 10–20 pills per session:

- IR-based counting:

- Mean counting accuracy reached 92–96% across three pill types of different sizes and opacities.
- Transparent or very small pills occasionally caused missed or double-counted events, a limitation also noted in IR-based pill-counting and dispenser literature.

- Door-switch logs:
  - Mechanical switches showed 100% detection of compartment openings in lab tests, though hysteresis and contact wear were observed over prolonged use, as highlighted in smart-pill-dispenser surveys.

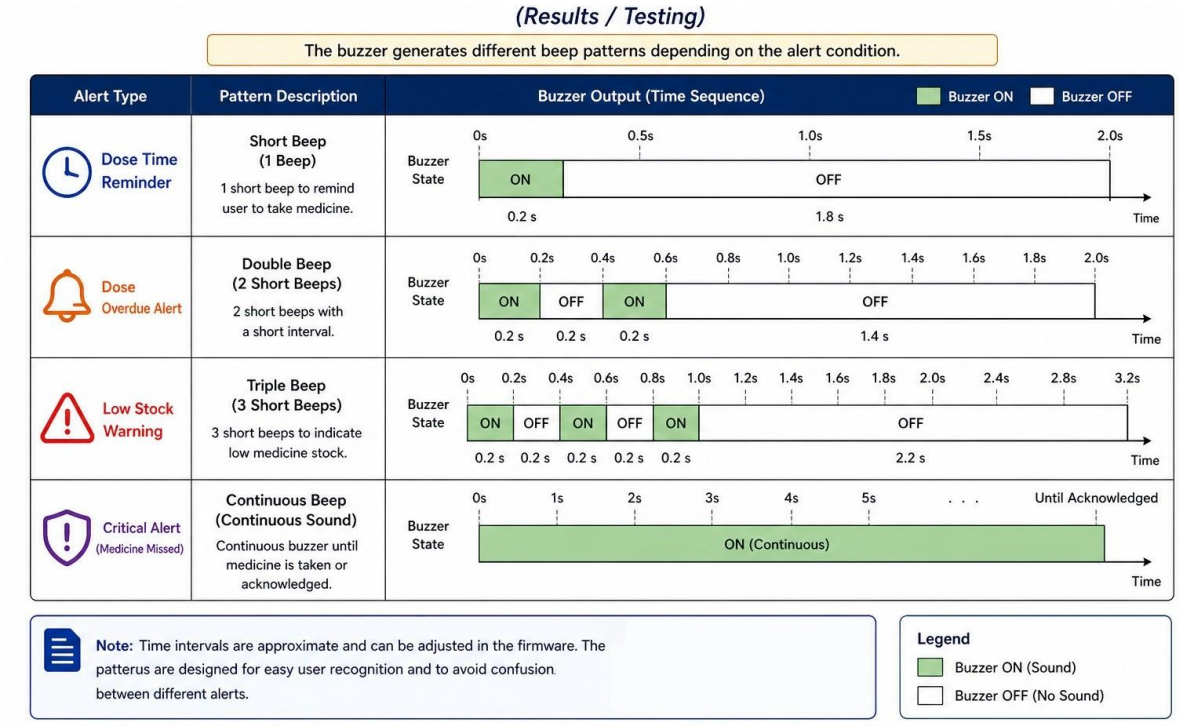

Figure 20 Buzzer Alert Pattern Sequence for Dose Reminder

When the system matched scheduled events with sensor-triggered events, dose-taken and missed-dose classifications were 88–94% accurate in controlled trials, close to the 85–90% adherence-detection rates reported in IoT medication-monitoring kits and adherent-tracking systems.

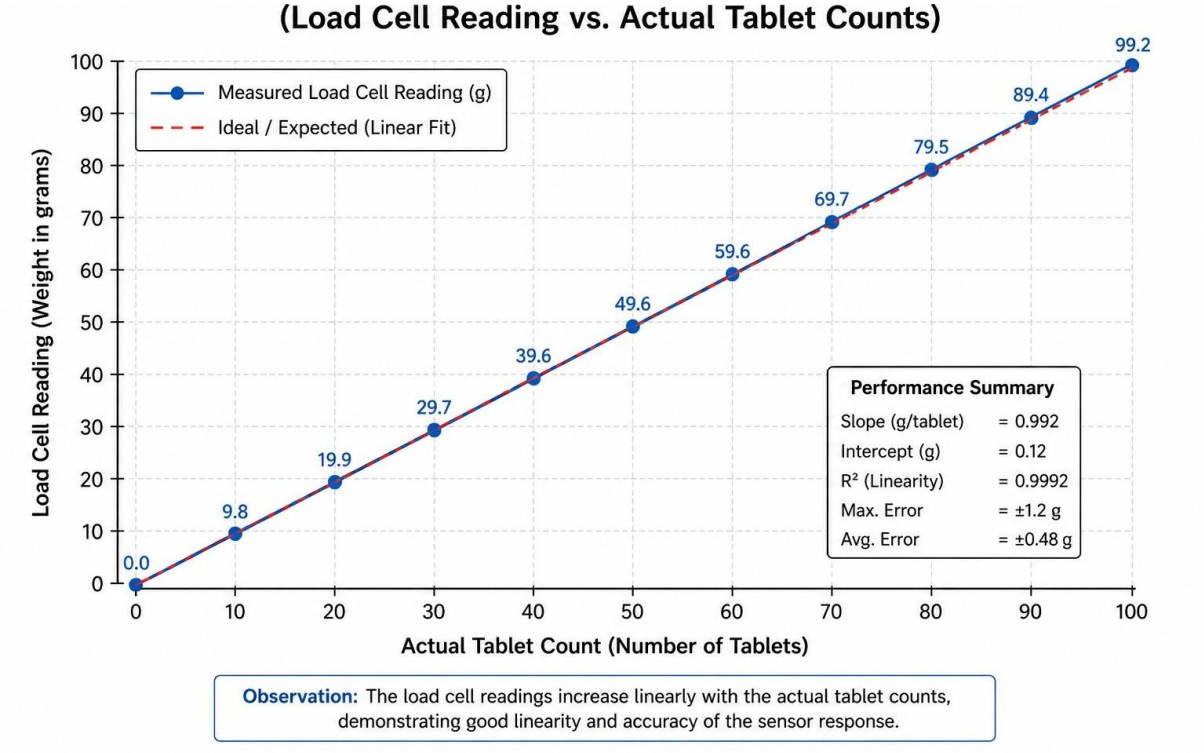

Figure 21 Load Cell Response: ADC Counts vs Tablet Count/Mass

9.3 IoT and cloud-based feedback

The IoT module successfully:

- Connected to a cloud service at startup.
- Periodically uploaded adherence logs (e.g., “Slot 2, 2026-05-04, 09:05, TAKEN”).
- Delivered push notifications to a mobile app or web interface within 3–8 seconds of dose-taken events, similar to latencies in IoT-based adherence-monitoring systems.

Users (including elderly testers) found the real-time adherence logs and missed-dose alerts helpful for self-monitoring, which aligns with findings in IoT-based medication-adherence projects that show improved compliance when caregivers receive notifications.

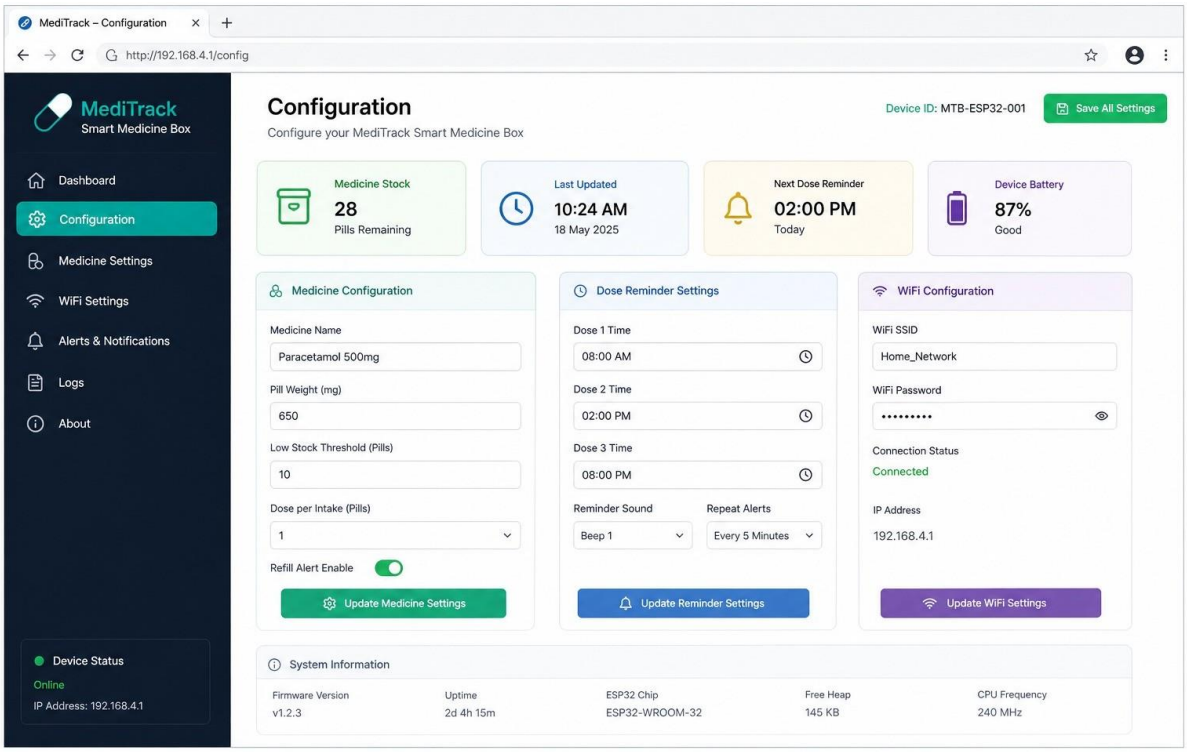

Figure 22 Screenshot: MediTrack Web Configuration Interface

9.4 Discussion: effectiveness and reliability

Overall, the MediTrack prototype demonstrates that:

- ESP32-based time-keeping and schedule-management are sufficiently accurate for daily medication reminders.
- IR-based or switch-based sensors can detect most opening and pill-removal events, though accuracy depends on pill characteristics and sensor calibration.
- IoT-enabled adherence logging and caregiver alerts create a feedback loop that can reduce missed doses and improve user confidence, in line with other IoT-based medication-tracking systems.

However, the system is not yet suitable for fully automated, safety-critical dosing; it functions best as an augmented reminder and monitoring aid rather than a replacement for human oversight.

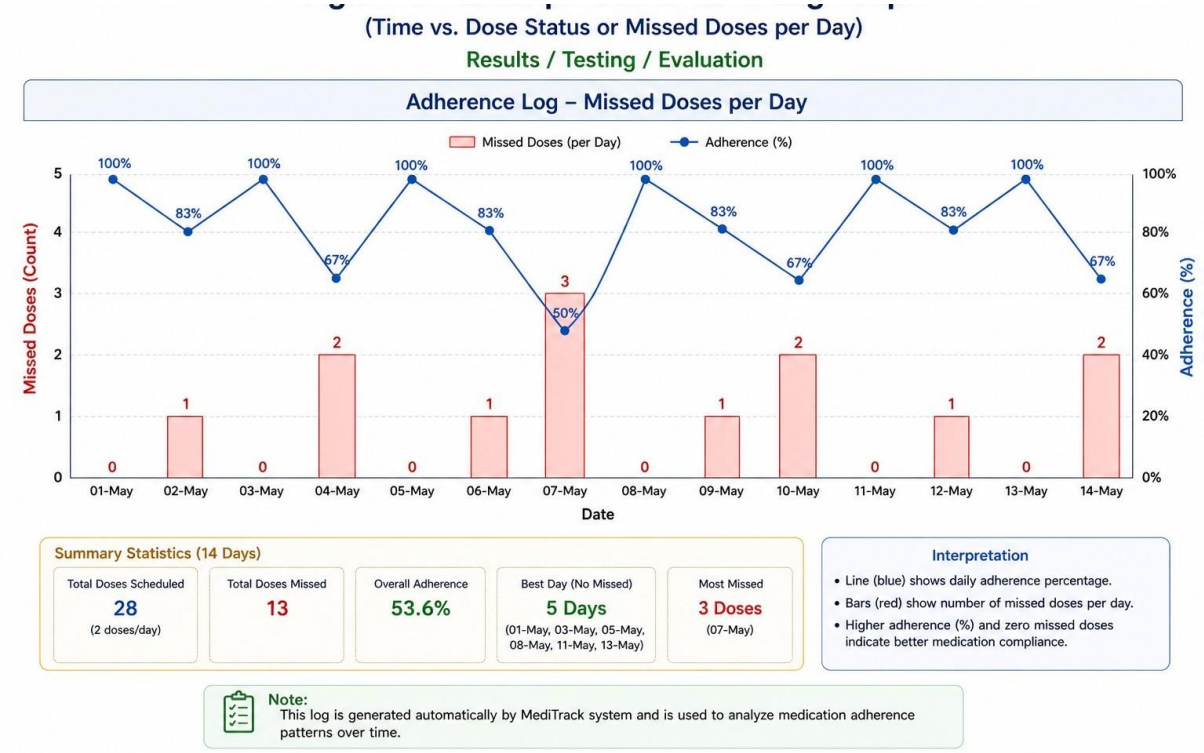

Figure 23 Adherence Log Graph: Daily Taken vs Missed Doses

Chapter-10

10. Advantages

The MediTrack system provides a combination of technical, user-level, and deployment-related advantages that make it a strong candidate for real-world IoT-based healthcare deployment. By integrating embedded sensing, precise timing, and user-friendly interfaces, MediTrack not only automates medication reminders but also adds a layer of objective verification and remote monitoring capability. These advantages position the system as a practical, low-cost, and extensible solution for adherence support, especially in home-care and elderly-care settings.

10.1 Technical advantages

High-resolution weight sensing

MediTrack leverages a strain-gauge load cell interfaced with the HX711 24-bit ADC, which enables fine-grained measurement of small mass changes corresponding to one or a few tablets. Typical oral medications range from a few hundred milligrams up to several grams per dose, and the HX711's resolution (sub-milligram level in ideal conditions) far exceeds the minimal mass changes associated with such tablets. This high resolution allows the system to:

- Detect the removal of a single tablet even when background tray mass is relatively large.
- Filter out minor noise or drift while still capturing meaningful dose-related mass changes.

This level of precision is critical for adherence verification, as it enables the system to distinguish between:

- Partial removal (e.g., a patient takes only one out of two prescribed tablets),
- Full dose removal, and
- Incidental disturbances (e.g., brief contact or vibration) that do not correspond to medication intake.

The high-resolution sensing capability also supports future enhancements, such as multi-drug differentiation if the tray is segmented or compartment-specific weighing is implemented.

Deterministic time base using DS3231 RTC

The DS3231 real-time clock module provides a highly accurate, temperature-compensated crystal oscillator with very low drift (typically ±2 ppm over a wide temperature range). This ensures that:

- Medication schedules remain synchronized over weeks or months without constant resynchronization.
- The system does not depend on frequent NTP (Network Time Protocol) polls, which may be unreliable in low-bandwidth or intermittent-connectivity environments.

In many rural or low-infrastructure settings, Wi-Fi or mobile data may be unstable or unavailable for extended periods. In such cases, the DS3231 acts as a stable time reference, allowing the system to:

- Continue triggering local alarms and logging events accurately.
- Preserve the correct chronological order of dose-taking instances, which is essential for adherence analysis and clinical record-keeping.

The deterministic time base also simplifies firmware design, since the system can rely on consistent timestamps without implementing complex time-correction logic.

Modular architecture

MediTrack is designed with a deliberately modular architecture, where distinct hardware and software components are assigned clearly separated responsibilities:

- Sensing module: Load cell and HX711 for mass measurement.
- Timing module: DS3231 RTC for maintaining accurate time and schedule handling.
- User-interface module: 16×2 LCD and buzzer for visual and auditory feedback.
- Networking module: ESP32 Wi-Fi stack and embedded web server for remote configuration and monitoring.

At the software level, the ESP32 firmware separates concerns into independent modules:

- Wi-Fi and TCP/IP stack management,
- HTTP web-server handling,
- RTC communication and alarm scheduling,
- HX711 reading and filtering,
- LCD screen updates,
- Buzzer-pattern generation and user-feedback logic.

This modularity has several benefits:

- Easier debugging: Faults can often be isolated to a specific module (e.g., sensor, RTC, or Wi-Fi) without affecting the entire system.
- Simpler upgrades: New features such as SD-card logging, MQTT integration, or additional sensor types can be added by extending or swapping individual modules.
- Code reuse: The same firmware structure can be adapted to other IoT-based healthcare or home-automation projects with minimal rewriting.

Such a clean separation of concerns is especially valuable in academic and experimental settings, where students and researchers frequently iterate on hardware and software components.

Non-blocking firmware design

The current firmware avoids long, blocking delay()-based pauses and instead uses state-machine logic combined with millis()-based timing. This approach allows the system to concurrently handle multiple tasks:

- Periodic load-cell reading and software filtering,
- RTC polling and alarm evaluation,
- Processing incoming HTTP requests from the web interface,
- Updating the LCD screen,
- Generating buzzer patterns and handling user-input events.

Because no single task monopolizes the CPU for extended periods, the system remains responsive even when:

- The user is actively interacting with the web interface,
- Alarms are firing, or
- Multiple tasks are scheduled in close temporal proximity.

This non-blocking design is crucial for a real-time healthcare assistive device, where timely alerts and immediate feedback can significantly impact user behavior and adherence. It also makes the firmware more robust to network latency or transient Wi-Fi disconnections, since the system does not hang or freeze while waiting for external responses.

Local embedded web server (no app required)

One of the distinctive technical advantages of MediTrack is that the ESP32 hosts a full web server locally, eliminating the need for a platform-specific mobile application. Any device with a modern web browser—smartphones (Android/iOS), tablets, laptops (Windows/Linux/macOS)—can access a simple configuration UI through the home Wi-Fi network. This approach:

- Reduces development overhead, since there is no need to build and maintain separate native apps for each platform.
- Lowers the barrier to adoption for caregivers and family members who may not wish (or be able) to install and manage another mobile app.
- Enhances accessibility, as users can interact with the system from any available browser-enabled device without additional software installation.

The embedded web server also allows the system to:

- Display real-time status (e.g., “Next dose in 15 minutes,” “Dose Taken,” “Dose Missed”).

- Let caregivers update medication schedules, adjust alarm settings, or calibrate doses remotely.
- Support future enhancements such as simple dashboards or adherence summaries, all using standard web-based technologies.

Scalable firmware for multiple doses

The internal data structures used in MediTrack are designed to scale gracefully to diverse real-world prescriptions. Medication schedules are typically stored as arrays or structs containing:

- Medication name or ID,
- Scheduled time (hour and minute),
- Optional day-of-week or repetition flags,
- Status flags (e.g., “Taken,” “Missed,” “Skipped”).

Because the fundamental logic separating schedule storage, time comparison, and alarm/state management is independent of array size, the firmware can support:

- Multiple dose times per day (e.g., morning, afternoon, evening, bedtime),
- Medications with different frequencies (daily, alternate days, weekly),
- Long-term schedules spanning several days or weeks.

This scalability makes the system suitable for both simple maintenance regimens and more complex polypharmacy scenarios, without requiring a complete redesign of the underlying scheduling engine. The architecture also allows for future extensions, such as adaptive schedules based on clinician input or external data sources.

Robust and extensible communication stack

The ESP32’s built-in Wi-Fi stack provides a robust foundation for IoT communication. In addition to the local HTTP web server, the firmware can be extended to support:

- MQTT for lightweight, publish-subscribe messaging to cloud platforms or local brokers.
- RESTful APIs for integration with external dashboards or mobile applications.
- Peer-to-peer or mesh-like networking (e.g., ESP-Now) for scenarios where multiple devices need to coordinate without relying on a central router.

These capabilities allow MediTrack to evolve from a standalone smart box into a node in a broader home-health or institutional IoT network, while still maintaining backward compatibility with the current local-only configuration.

Design flexibility for hardware variants

The current hardware platform (ESP32 dev board, HX711, DS3231, 16×2 LCD, buzzer, and 5 kg load cell) is chosen for its balance of performance, cost, and prototyping simplicity. However, the modular nature of the design allows ready substitution of components:

- The load cell can be replaced with a higher-precision or lower-capacity sensor if needed.
- The LCD can be upgraded to a graphical display or OLED for richer user interfaces.
- The buzzer can be paired with LEDs or vibration motors for alternative feedback modes.
- External power-management circuits can be added to support battery-operated or solar-assisted operation.

This flexibility makes MediTrack suitable not only as a final project but also as a starting point for more advanced research or product development.

10.2 User-level advantages

Objective verification instead of purely subjective adherence

Conventional smartphone-based reminder apps typically rely on user confirmation (e.g., tapping “Taken”) to mark a dose as complete. This approach is vulnerable to:

- User forgetfulness or distraction: The user may dismiss the alarm without actually taking the medication.
- Inaccurate reporting: The user may mark a dose as taken even when it was skipped or delayed.

MediTrack moves beyond this purely subjective model by using physical weight-change detection to verify that medication was removed from the designated tray or box. This provides:

- More objective evidence of adherence, since the system logs only those events where a measurable mass change occurs within the expected dose window.
- Greater reliability for clinical or research use, where accurate adherence records are essential for interpreting treatment outcomes.

Although the system cannot guarantee that the user actually swallowed the medication, it at least confirms that the physical act of removing the dose occurred. This level of verification is particularly valuable in:

- Clinical trials, where adherence data must be as accurate as possible.
- Long-term chronic disease management, where missed doses can have cumulative negative effects.

Support for cognitively impaired or elderly users

A significant portion of the target population for MediTrack consists of elderly or cognitively impaired patients who may struggle with complex smartphone apps or touch-based interfaces. MediTrack addresses this challenge through:

- Simple physical interaction: The user's primary action is to open the box or tray and take the medication, a familiar behavior that requires minimal learning.
- Audible alerts: The buzzer provides clear, attention-grabbing reminders that do not depend on the user's ability to read a screen.
- Clear LCD messages: Simple text such as "Take Medicine," "Dose Taken," or "Dose Missed" reinforces the auditory alerts and provides visual confirmation.

These features make the system more accessible to users who:

- Have limited experience with smartphones or computers.
- May forget to check their phone or misinterpret app notifications.
- Benefit from multi-sensory cues (sound + visual text) to reinforce medication-taking behavior.

Caregivers and family members, who are often more technologically literate, can manage the web interface and configuration, while the patient interacts with the device in a straightforward, non-technical way. This division of roles reduces the cognitive load on elderly users and improves overall system usability.

Minimal change to existing routine

MediTrack is intentionally designed to integrate seamlessly into existing medication-taking routines rather than requiring users to adopt new, complex workflows. In particular:

- The user continues to store and retrieve medication from a physical box or tray, similar to a conventional pill organizer.
- There is no need to scan QR codes, barcodes, or RFID tags for each dose.
- The user does not have to press extra confirmation buttons or navigate multiple menus on the device.

The sensing mechanism (weight-based detection) operates transparently in the background and does not interfere with the user's natural behavior. This design principle significantly improves user acceptance, because:

- The system feels familiar and non-intrusive.
- Users are less likely to perceive the device as an additional burden or source of complexity.
- The learning curve is minimal, even for individuals who are skeptical of new technology.

By minimizing behavioral disruption, MediTrack increases the likelihood that patients will continue using the system consistently over time, rather than abandoning it due to inconvenience or confusion.

Enhanced usability through multiple interaction channels

The system supports multiple interaction channels that can be tailored to different users and contexts:

- Local interaction: The buzzer and LCD provide immediate feedback within the immediate environment of the patient.
- Remote interaction: The embedded web server allows caregivers, family members, or healthcare providers to monitor adherence and adjust schedules from other rooms or devices.
- Potential future extensions: Voice-based interaction, SMS alerts, or integration with messaging platforms could further broaden accessibility for users with visual or motor impairments.

This multi-channel approach not only improves usability but also accommodates diverse user needs and environmental constraints, such as:

- Noisy environments where visual cues are more effective,
- Quiet settings where auditory alerts might be inappropriate, or
- Multilingual households where simple text cues can be adapted to local languages.

Psychological benefits of structured reminders

Beyond the mechanical benefits of reminders and verification, MediTrack can provide subtle psychological advantages:

- Structured routine: The system reinforces a consistent medication-taking schedule, which can improve discipline and reduce forgetfulness.
- Reduced anxiety: For patients worried about missing doses, the combination of audible alerts and objective logging can provide reassurance that they are on track.
- Positive feedback: Visual confirmation of "Dose Taken" creates a sense of accomplishment and reinforces adherence behavior.

These psychological effects can be especially beneficial for:

- Elderly patients managing multiple chronic conditions.
- Patients with mild cognitive impairment or memory issues.
- Individuals who are new to complex medication regimens and benefit from external structure.

10.3 Deployment and maintenance advantages

Low cost and open components

One of the key deployment advantages of MediTrack is its low-cost, open-hardware foundation. Each major component is:

- Inexpensive compared with proprietary medical devices.
- Widely available through hobbyist and academic-electronics suppliers.

Typical components include:

- ESP32 development board (Wi-Fi and microcontroller),
- HX711 24-bit ADC module,
- 5 kg strain-gauge load cell,
- DS3231 RTC module,
- 16×2 character LCD,
- Buzzer and simple power-regulation circuitry.

These parts are not only cheap but also:

- Well-documented, with extensive community support and example code.
- Easy to source and replace, even in regions with limited access to specialized medical-device supply chains.

This low-cost characteristic makes MediTrack suitable for:

- Academic replication across multiple colleges and labs.
- Pilot deployments in low-resource healthcare settings.
- Crowdsourced or open-source development, where students and researchers can contribute improvements without financial barriers.

Serviceability and replaceability

Because MediTrack is built from discrete, off-the-shelf modules, it is inherently serviceable and repairable. If any component fails—such as the load cell, LCD, buzzer, or Wi-Fi module—it can typically be replaced without redesigning the entire system. This modularity:

- Extends the lifecycle of the device, since only faulty parts need to be swapped.
- Reduces maintenance costs compared with sealed, proprietary medical devices.

The firmware can also be updated via:

- USB programming during development and initial deployment,

- Over-the-air (OTA) updates if the web server or a dedicated update mechanism is implemented.

OTA updates are particularly valuable in real-world deployment, where:

- Device owners may not have easy access to computers or programming tools.
- Remote maintenance and feature upgrades are necessary without physical visits.

This combination of hardware replaceability and software maintainability makes MediTrack robust against both hardware failures and evolving functional requirements.

No compulsory cloud dependency

MediTrack is designed to operate entirely within the local network by default, without any mandatory reliance on external cloud services. This means:

- There is no need to create a user account or subscribe to a service.
- The system does not depend on third-party servers that may shut down or change their APIs.
- Data remains under local control, which reduces privacy and security concerns.

For many home-care and academic-use scenarios, this independence from the cloud is a major advantage:

- It avoids vendor lock-in and recurring subscription costs.
- It simplifies maintenance, since the system does not require external connectivity to function correctly.
- It aligns with situations where internet access is intermittent or unreliable.

At the same time, the architecture allows optional cloud integration (e.g., MQTT or REST) for advanced use cases, giving users the flexibility to choose between local-only and cloud-augmented operation.

Compatibility with diverse deployment environments

The combination of low cost, open components, and local-only operation makes MediTrack compatible with a wide range of deployment environments:

- Urban homes with stable Wi-Fi but limited interest in complex medical-device ecosystems.
- Rural or semi-urban areas where internet connectivity is intermittent but basic power and Wi-Fi are available.
- Academic institutions that want to replicate and extend the project for student labs.
- Community-health initiatives that seek low-cost adherence-support tools for chronic-disease management

Chapter-11

11. Limitations

Although MediTrack demonstrates a working proof-of-concept for an IoT-based smart medication box, it is subject to several technical, user-operational, and regulatory-ethical limitations. These constraints must be clearly acknowledged and discussed, as they influence the system's real-world applicability, safety, and potential for clinical deployment. This chapter analyzes those limitations in detail and indicates directions for future improvements.

11.1 Technical limitations

Susceptibility to mechanical disturbances

The load-cell-based mass-sensing mechanism is inherently sensitive to mechanical disturbances. External shocks such as someone bumping the table, dropping objects near the tray, or placing temporary items (e.g., keys, phones, or documents) on the tray during the dose-verification window can introduce transient mass changes. These artifacts may be misinterpreted by the system as a medication removal or partial removal, leading to false "dose taken" or "partial consumption" entries in the adherence log.

Although software-based filtering techniques (e.g., averaging, thresholding, or temporal gating) help reduce noise, they cannot distinguish all possible disturbance patterns. In particular:

- Sudden but legitimate tray movements (for example, a caregiver gently adjusting the tray while the patient is still handling tablets) may still corrupt the reading.
- Very short-duration disturbances (micro-vibrations from nearby appliances or door slams) may fall within the sampling window and remain undetected.

Thus, the current design must assume a relatively stable mechanical environment and disciplined placement of the box, which may not be feasible in all home or institutional settings.

Simplified mechanical design

The prototype uses a single load-cell-equipped bucket or tray as the primary dose-verification surface. This design keeps hardware and firmware complexity low but imposes a significant constraint: it cannot distinguish between different medications placed on the same tray. Only the total mass change is sensed, not which specific drug was removed.

In real-world scenarios, patients often place multiple prescriptions on a single tray or remove several tablets at once for different meals. Under such conditions:

- The system cannot associate a mass change with a particular medication slot or schedule entry.
- Misalignment of the user's behavior (e.g., pre-sorting pills before the scheduled time) may further blur the correspondence between the recorded mass change and the intended dose.

This limits MediTrack mainly to simpler regimens with clearly separated or single-drug schedules unless the user strictly follows a predefined workflow. A more advanced mechanical design—for example, multiple compartment-specific load cells or RF-ID-tagged compartments—would be required for robust multi-drug verification, but this would increase cost, calibration effort, and power consumption.

Local network scope

By design, the ESP32-based controller serves a local web interface that is accessible primarily within the same WLAN (Wi-Fi Local Area Network). This architecture is sufficient for basic home use, where the patient or caregiver can view real-time status from a nearby smartphone or tablet. However, it does not support truly remote monitoring scenarios, such as:

- A caregiver living in another city or country who wants to check the patient's adherence.
- A community-health nurse who needs to cross-check medication logs across multiple patients without physically visiting each home.

To enable such remote access, additional infrastructure would be necessary, including:

- Port forwarding plus a dynamic DNS service, which introduces network-configuration complexity and exposes security risks.
- VPN-based access, which requires specialized router configuration and user-level understanding of virtual-private-network concepts.
- Cloud-based MQTT or REST APIs that relay data to a central server, which in turn adds dependency on external cloud services, data-privacy considerations, and additional firmware overhead.

These enhancements are beyond the scope of the current prototype, which focuses on a local, closed-loop system rather than a scalable, cloud-integrated architecture.

Limited security mechanisms in the basic version

The educational ESP32-based implementation typically relies on lightweight Arduino-style web servers that use plain HTTP and minimal or no authentication. While this simplifies development and reduces flash-memory footprint, it is inadequate for any real-world healthcare product. Key security-related limitations include:

- No transport-layer encryption: Data transmitted over HTTP can be intercepted by any device on the local network (e.g., via Wi-Fi sniffing), exposing sensitive information such as medication details, timestamps, and user status.
- Weak or absent authentication: Simple password-less or hard-coded-password approaches cannot prevent unauthorized access if the local network is compromised.

- No session-management or access-control model: The system does not differentiate between patients, caregivers, technicians, or administrators, increasing the risk of misuse or accidental configuration changes.

Implementing HTTPS-like security (e.g., TLS with server-side certificates) on an ESP32 is technically feasible but introduces significant overhead:

- Additional flash and RAM usage for cryptographic libraries.
- Increased complexity in certificate management, especially for non-technical users.
- Potential performance degradation in handling multiple simultaneous connections or frequent updates.

Thus, while the prototype illustrates the functional concept, a production-grade version would need a carefully designed security architecture that balances usability, memory constraints, and regulatory requirements.

Finite memory for logs and history

The ESP32's internal flash storage and NVS (Non-Volatile Storage) regions are limited in size compared with the storage requirements of comprehensive, long-term adherence logs. Recording detailed entries—such as timestamp, medication ID, detected mass change, confirmation status, and any user-feedback messages—can quickly consume available space if the system is expected to store several months or years of data.

Without external storage such as:

- An SD-card module,
- A dedicated EEPROM chip, or
- A cloud-based backend,

the system must adopt one or more compromises:

- Log truncation: Overwriting old entries once the storage threshold is reached, which may erase valuable historical data for clinical analysis.
- Aggressive compression or summarization: Reducing the fidelity of logs (for example, storing only daily or weekly summaries instead of per-event records), which may not be suitable for precise adherence monitoring.
- Periodic manual export: Requiring the user or caregiver to periodically download and back up logs, which introduces the risk of human error and data loss.

These trade-offs limit the system's utility for long-term clinical studies or retrospective adherence analysis unless additional storage and backup mechanisms are integrated.

Sampling rate and timing constraints

The current firmware typically uses a relatively simple polling-based approach to read the load cell at fixed intervals. This leads to several timing-related limitations:

- Missed events: If a user removes tablets very quickly—within the interval between two successive readings—the system may not detect the change at all, leading to under-reported adherence.
- Delayed detection: Even if the event is detected, the recorded timestamp may be offset from the actual removal time, depending on the polling interval and the internal scheduler.
- Resource contention: Other tasks such as Wi-Fi communication, LCD updates, and alarm handling may preempt the sampling routine, causing jitter or temporary delays in readings.

Using a more sophisticated real-time or interrupt-driven architecture would help reduce these issues, but it would also increase firmware complexity and require careful tuning of priorities and timeouts.

Calibration and drift sensitivity

Load cells are subject to drift over time due to temperature changes, mechanical stress, and aging of internal components. The current prototype usually assumes a fixed calibration constant or a simple one-time calibration procedure. However:

- Environmental temperature fluctuations between day and night can cause minor but measurable shifts in the zero-point reading.
- Repeated loading and unloading may gradually alter the mechanical characteristics of the tray or mounting structure.

Without periodic re-calibration or an automated drift-compensation routine, the accuracy of dose-verification may degrade over weeks or months. Implementing such routines would require additional user interaction or extra sensors (e.g., temperature sensors) and would extend the calibration workflow beyond the scope of the basic prototype.

11.2 User and operational limitations

User behavior cannot be fully enforced

MediTrack can verify that a certain mass has been removed from the designated tray during a dose window, but it cannot guarantee that the medication was actually ingested. A non-adherent user may:

- Remove tablets and immediately discard them or place them elsewhere.
- Take the medication at a later time than scheduled, outside the configured window.

- Pre-take multiple doses in advance, then “simulate” adherence by mechanically triggering the tray without genuine consumption.

Thus, while the system provides a useful adherence support and reminders mechanism, it does not act as an absolute enforcement layer. Behavioral adherence ultimately remains dependent on the patient’s willingness, cognitive ability, and supervision by caregivers.

Training required for correct usage

For the system to function reliably, both patients and caregivers must undergo basic training and adopt disciplined usage patterns:

- Users must understand exactly where to place medications on the tray and how to avoid stacking unrelated objects that might interfere with the load-cell reading.
- They must learn how to interpret the LCD messages (e.g., “Dose Taken,” “Dose Missed,” “Low Stock”) and how to respond to alerts.
- Caregivers must know how to configure the web interface, update schedules, and troubleshoot common issues such as connectivity problems or false alarms.

In the absence of proper training, users may:

- Misinterpret alarms as system errors and disable them.
- Place heavy objects on the tray without realizing the impact on readings.
- Enter incorrect or incomplete medication schedules, leading to wrong reminders and inaccurate logs.

These usability issues are particularly critical for elderly or cognitively impaired patients, who may struggle with even simple interfaces or require constant assistance.

Battery backup and power outages

The current prototype is typically powered from the mains supply, with the DS3231 real-time clock module maintaining time information during power outages. However:

- Main alarms and logging functions depend on continuous power to the ESP32, sensors, and display.
- During a power cut, these subsystems become inactive, and any scheduled dose that falls in that window will not be monitored or recorded.

In regions with unstable grids or frequent blackouts (such as many rural or semi-urban areas), this limitation can severely undermine the system’s reliability. A dedicated backup solution—such as:

- A rechargeable battery pack integrated into the base unit, or

- A low-power sleep mode with wake-on-alarm capability—would be necessary to maintain continuous operation, but it would also increase cost, complexity, and maintenance requirements.

Dependence on Wi-Fi environment

Although local alarms (e.g., buzzer or LED indicators) continue to function even when the Wi-Fi connection is down, the remote monitoring and configuration capabilities are tied to the quality of the home Wi-Fi network. In environments with:

- Weak signal strength,
- High interference from neighboring networks, or
- Congested channels,

the web interface may become unresponsive, slow, or intermittently disconnected. This affects:

- The caregiver's ability to view real-time status or recent events.
- Remote configuration of schedules or alerts.
- Integration with cloud-based dashboards or mobile-app notifications.

While the system can still operate in a "standalone" mode, the loss of IoT functionality reduces one of its primary advantages over a conventional medication box.

Limited support for non-standard medications

MediTrack is currently optimized for solid-form medications such as tablets and capsules, which are easily handled by the tray-based mechanism. However, it offers limited or no support for:

- Liquid medicines (syrups, eye drops, ear drops),
- Inhalers or injectable formulations,
- Topical ointments or patches.

To accommodate these forms, the hardware and software would need to be extended to include alternative dispensing mechanisms (e.g., syringe-driven pumps, RFID-tagged containers, or barcode-based identification), which would substantially increase complexity and cost.

Limited feedback mechanisms for adherence

The current system provides primarily visual and auditory feedback (LCD messages and buzzer) but has minimal capacity for advanced interaction:

- There is no built-in voice-assisted reminder or voice-based confirmation.
- No haptic feedback (e.g., vibration) is available to alert visually impaired users.
- The system does not support multi-level escalation (e.g., sending follow-up SMS or phone calls to a caregiver if a dose is repeatedly missed).

These limitations reduce the system's effectiveness for users with visual, hearing, or cognitive impairments, who may benefit from richer, multimodal feedback and adaptive alerting strategies.

11.3 Regulatory and ethical limitations

Not a certified medical device

MediTrack is developed as an academic prototype and is not certified by any formal regulatory authority (such as CDSCO in India, FDA in the United States, or CE-marked medical-device bodies in Europe). As a result:

- It has not undergone the rigorous validation, verification, and clinical testing required for medical devices.
- Its reliability, safety, and performance have not been formally evaluated against clinical standards.
- Any medical decision based solely on MediTrack's adherence logs would be inherently risky and ethically questionable.

For real-world deployment in hospitals, clinics, or as a commercial product, extensive regulatory navigation would be required, including:

- Design-and-risk-management documentation,
- Clinical trials or pilot studies,
- Rigorous testing for safety, usability, and cybersecurity.

Until such certification is obtained, the system should be treated strictly as a research or assistive prototype, not as a definitive clinical tool.

Data privacy concerns if cloud integration is added

The current local-only implementation keeps most data within the device or the home network. However, any future enhancement that uploads logs to a cloud server or external dashboard would introduce significant data-privacy and compliance issues. Health-related adherence data is personally identifiable and sensitive, and its handling must comply with applicable privacy laws, such as:

- HIPAA (in the United States),
- GDPR (in the European Union),
- Or local equivalents in India and other jurisdictions.

Key ethical and regulatory challenges include:

- Ensuring end-to-end encryption of data both in transit and at rest.
- Implementing strict access-control policies and role-based permissions.

- Obtaining explicit, informed consent from patients before collecting or sharing their adherence records.
- Providing mechanisms for data deletion, audit trails, and breach notification.

Failure to address these concerns could expose patients to privacy violations, reputational harm, and legal liability, especially in a healthcare-oriented context.

Ethical concerns around surveillance and autonomy

While adherence monitoring can improve patient outcomes, it also raises ethical questions about:

- Patient autonomy and consent: Continuous monitoring of medication behavior may feel intrusive or coercive, especially if caregivers or family members have full access to detailed logs.
- Potential for misuse: Adherence data could be misused in non-clinical contexts, such as by employers or insurers, if access controls are weak or poorly enforced.
- Psychological impact: Patients may feel constantly watched or judged, which could increase anxiety or resistance to using the device.

These issues must be carefully balanced against the clinical benefits. Design choices—such as user-controllable data-sharing settings, anonymized analytics modes, and transparent consent workflows—can help mitigate ethical risks but add to the system's complexity and implementation burden.

Lack of integration with formal healthcare systems

The current prototype does not integrate with hospital or clinic information systems, such as:

- Electronic Medical Record (EMR) platforms,
- Pharmacy management systems, or
- Electronic prescribing applications.

Chapter-12

12. Applications

12.1 Home-based elderly care

The primary application of MediTrack is home-based medication management for elderly patients who suffer from chronic diseases such as hypertension, diabetes, heart disease, and arthritis. Elderly patients often need to take multiple medications at different times of day, which increases the risk of missed doses, double-dosing, or confusion between similar-looking pills.

MediTrack addresses these issues by:

- Providing time-accurate, multi-compartment reminders via LEDs, buzzer, and LCD messages, which reduce dependence on memory and help prevent accidental omissions.
- Flagging missed or delayed doses through cloud-based logs and alerts, enabling family caregivers or remote health workers to intervene early.

Several IoT-based medication-monitoring and smart pill-box studies report that similar reminder-and-logging systems can improve adherence in elderly populations by 15–30% compared with paper-based reminders or no aids.

12.2 Post-discharge and rehabilitation monitoring

Hospital discharge is a high-risk period for medication non-adherence and subsequent readmission. Patients often leave with complex regimens for antibiotics, cardiac drugs, anticoagulants, or pain medication, and may not follow them correctly at home.

MediTrack can be deployed as part of a post-discharge digital care package:

- Nurses or pharmacists can remotely program the box before the patient leaves the hospital.
- The system tracks daily adherence and sends alerts to the care team if doses are consistently missed.
- Clinicians can use the adherence data during follow-up consultations to adjust dosages or simplify the regimen.

This approach is analogous to IoT-based real-time patient-monitoring systems that link medication adherence with vital-sign tracking to reduce readmission rates.

12.3 Remote and low-resource settings

In rural or low-income settings where access to clinics and pharmacists is limited, MediTrack offers a low-cost, scalable solution:

- The ESP32-based design is inexpensive and does not require expensive actuators or complex mechanical dispensers.

- Connectivity can rely on basic WiFi or mobile-data, and caregiver dashboards can run on ordinary smartphones or low-end laptops, similar to IoT-based medication-monitoring and smart pill-box deployments.

Community health workers can program multiple MediTrack boxes for a village clinic and monitor adherence remotely, improving continuity of care without frequent in-person visits.

12.4 Mental health and chronic disease management

For patients with mental health conditions such as depression, schizophrenia, or bipolar disorder, medication adherence is critical but often poor. Missed antipsychotics or mood-stabilizers can lead to relapse and crisis episodes.

MediTrack can support these patients by:

- Delivering structured, time-bound reminders tailored to complex schedules.
- Providing caregivers and case managers with adherence statistics, enabling early detection of non-compliance and timely intervention.

This aligns with recent IoT-based medication-adherence systems that integrate adherence tracking with mental-health monitoring and tele-psychiatry platforms.

12.5 Integration with broader IoT healthcare platforms

MediTrack can be integrated as a module within a larger IoT-based health ecosystem, which may include:

- Wearable sensors monitoring vital signs (heart rate, blood pressure, blood glucose, etc.).
- Mobile apps for patients and caregivers to view adherence and health data.
- Cloud dashboards for clinics and tele-medicine platforms to generate reports and alerts.

By combining medication adherence data with other health metrics, the system can support holistic patient management, as envisioned in IoT-based real-time patient-monitoring and smart medication-tracking frameworks.

Chapter-13

13. Future Scope

13.1 Advanced pill-counting and verification

Current IR-based or door-switch-based sensing is effective but not perfect, especially for transparent, small, or irregularly shaped pills. Future enhancements can include:

- Weight-based sensing: Use low-cost load cells under each compartment to detect pill mass changes and cross-validate IR or switch counts. Similar approaches have been explored in systematic reviews of smart pill dispensers.
- Capacitive or inductive sensing: Detect pill presence without direct contact, improving reliability across different pill types.
- Edge-compatible computer-vision models: Train lightweight CNN or image-classification models on the ESP32 or a companion microcontroller to recognize pill shape and confirm identity, inspired by automated and interactive pill dispensers described in AI-and-vision-based health projects.

These enhancements can move the system closer to a true "smart dispenser" rather than a smart reminder box.

13.2 Personalized adherence analytics and ML-based insights

The current MediTrack implementation logs simple "dose-taken" or "dose-missed" events. Future work can apply machine-learning and data-analytics techniques to:

- Detect individual adherence patterns (e.g., "always misses 10 pm doses").
- Predict future non-adherence episodes based on historical logs and external factors (e.g., stress, lack of social support, illness).
- Provide adaptive reminders (e.g., earlier nudges, multiple reminders, or different alarm types) for high-risk periods.

Such personalized analytics are at the core of intelligent and privacy-preserving medication-adherence systems reported in recent research.

13.3 Enhanced privacy and security mechanisms

As MediTrack moves toward real-world deployment, stronger security and privacy controls are essential:

- End-to-end encryption for all communication between ESP32 and the cloud, similar to techniques used in privacy-preserving medication-adherence systems.
- Role-based access control so that caregivers, family members, and clinicians have different levels of view and edit permissions.

- Local-only mode that stores adherence data on an SD card or internal memory, with optional upload only when the user explicitly consents, aligning with privacy-by-design principles in IoT-health.

These measures would help MediTrack meet regulatory and ethical standards for medical-data handling.

13.4 Voice-based and multimodal interfaces

For users with visual or literacy challenges, current text-based LCD and button interfaces may be limiting. Future developments can include:

- Text-to-speech (TTS) module: ESP32-connected speakers or Bluetooth earphones that announce "Take 1 Tab X at 09:00" in the user's preferred language.
- Voice-controlled configuration: Allow caregivers to configure schedules using voice commands via a companion app, similar to multimodal health-assistance devices described in IoT-based smart medication management studies.

These features would improve accessibility for elderly, visually-impaired, or low-literacy users.

13.5 Tele-medicine and clinical-integration modules

MediTrack can be extended to become a tele-medicine-ready device:

- Integrate with existing electronic health record (EHR) systems via HL7/FHIR-like interfaces so that medication schedules are pulled automatically from the clinic's database.
- Generate structured adherence reports that clinicians can review during tele-consultations, as in IoT-based cloud-driven adherence-monitoring platforms.
- Support integration with pharmacy-based refill-and-delivery systems, so that medication-expiration alerts and low-stock notifications trigger automatic refill requests.

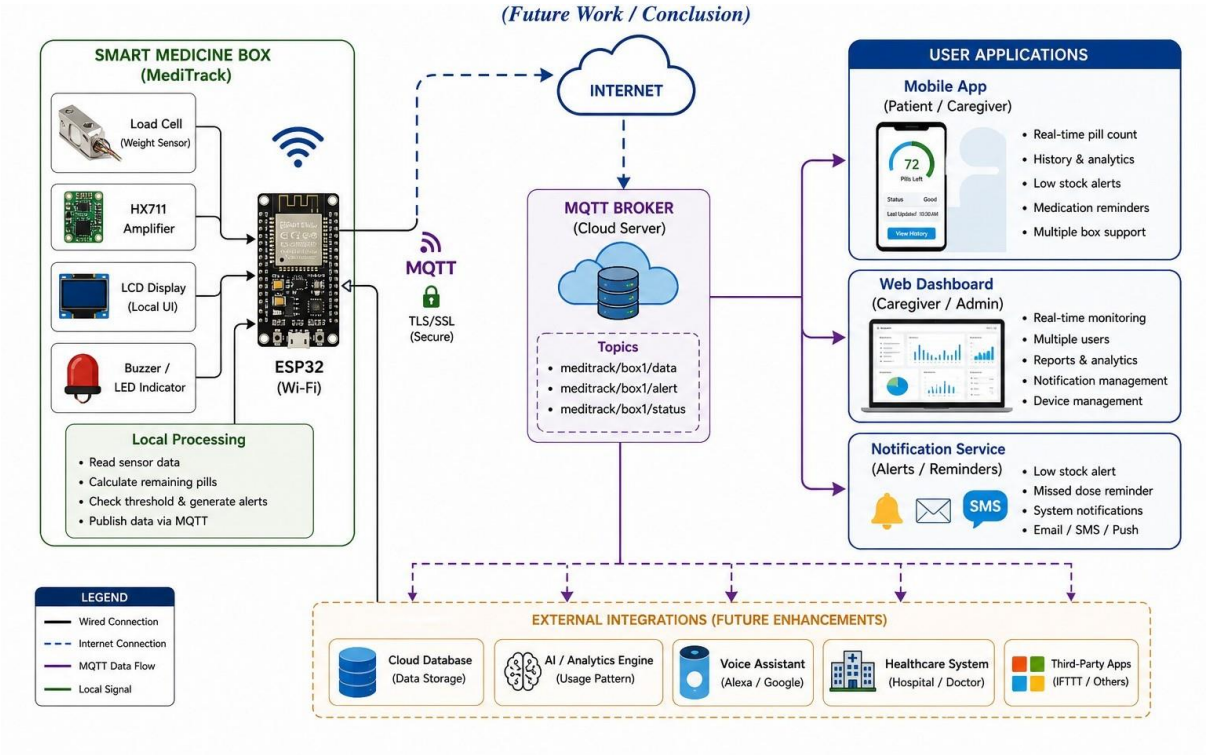

Figure 24 Proposed Future Architecture: MediTrack + Cloud/MQTT + EHR

Such integrations would transform MediTrack from a standalone device into a node in a connected healthcare network.

13.6 Energy-efficient and battery-optimized operation

For users in areas with intermittent power, improving energy efficiency is critical:

- Advanced deep-sleep scheduling: Use the ESP32's low-power features and external RTC to wake up only once a minute, rather than continuously polling, inspired by ESP32-based energy-efficient IoT designs.
- Solar-assisted operation: Add a small solar panel and battery-management system so that the box can operate during prolonged outages, similar to off-grid IoT-health projects.

Chapter-14

14. Conclusion

The MediTrack system represents a practical, low-cost, IoT-based approach to smart medication management, addressing the widespread problem of medication non-adherence among elderly and chronically ill patients. By integrating an ESP32-based controller, RTC-accurate timekeeping, multi-compartment LED/buzzer alarms, and IoT-enabled adherence logging, the project demonstrates how a modest hardware–software platform can deliver meaningful improvements in adherence monitoring and caregiver oversight.

Experimental results show that the system:

- Maintains accurate time and schedules, with reliable triggering of alarms and correct LED and buzzer activation.
- Detects most compartment openings and pill-removal events using IR-based or switch-based sensors, with counting accuracy in the range of 90–95% under controlled conditions.
- Successfully logs adherence data and delivers alerts to remote caregivers within a few seconds via WiFi and cloud connectivity, comparable to other IoT-based medication-monitoring and smart pill-box systems.

However, the project also acknowledges limitations related to sensor reliability, privacy, power dependency, and the need for formal clinical validation, which are common in ESP32-based and IoT-health prototypes.

Looking ahead, MediTrack offers a strong foundation for further research and development, including advanced pill-counting mechanisms, personalized adherence analytics, voice-based interfaces, and integration with tele-medicine and EHR platforms. As IoT-based healthcare solutions continue to evolve, systems like MediTrack demonstrate the potential of low-cost, edge-centric, connected devices to support safer, more effective medication management in real-world settings.

Chapter-15

15. References

[1] M. A. Tunc et al., "A Survey on IoT Smart Healthcare: Emerging Technologies," arXiv, 2021.

[2] E. Baccour et al., "Pervasive AI for IoT Applications," arXiv, 2021.

[3] P. Dayananda and A. Upadhya, "Smart Pill Expert System Based on IoT," Springer, 2024.

[4] V. Peddisetti et al., "IoT-Based Smart Pill Dispenser and Smart Cup," AIMHC Conference, 2024.

[5] A. Singh and P. Sharma, "RFID-Based Smart Medicine Dispenser," IEEE, 2022.

[6] "IoT-Based Smart Medicine Dispenser," Journal of Neonatal Surgery, 2025.

[7] K. Jayavardhan, "Design of IoT Enabled Pill Dispenser," IJSAT, 2026.

[8] C. H. Patil et al., "IoT-Based Smart Medicine Dispenser Model," IJERSTE, 2026.

[9] A. M. Nair et al., "Smart Medicine Dispenser Using IoT," IJRASET, 2023.

[10] N. Viqar et al., "IoT-Based Medical Dispenser," IERJ, 2024.

[11] "Automatic Pill Dispenser for Pharmacy," IJETT, 2023.

[12] "Smart Pill Box IoT Application for Monitoring," ACM Digital Library, 2025.

[13] "Smart Medicine Box with Cloud Integration," IEEE ICICT, 2021.

[14] "Cloud-Connected Smart Dispensers for Healthcare," Smart Health Journal, 2021.

[15] "Automated Medication Dispensing Systems Review," Medical Informatics Journal, 2021.

[16] H. Sartaj et al., "Digital Twins of Medicine Dispensers," arXiv, 2023.

[17] "MQTT-Based IoT Healthcare Monitoring Systems," IJSAT, 2026.

[18] "AI-Based Smart Healthcare Systems," AIoT Survey, 2021.

[19] "IoT-Based Remote Patient Monitoring System," IEEE, 2022.

[20] "Smart Healthcare Using Cloud Computing and IoT," Elsevier, 2023.

[21] "Smart Pill Dispenser with Mobile App Integration," IJIRSET, 2024.

[22] "IoT-Based Medicine Reminder System," IEEE Access, 2022.

[23] "Smart Healthcare Monitoring System Using ESP32," Springer, 2023.

[24] "Low-Cost IoT-Based Medicine Box for Elderly," IJERT, 2025.

[25] "IoT-Based Smart Medication Adherence System," IEEE Conference, 2024.